

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**TÀI LIỆU HƯỚNG DẪN THỰC HÀNH
MÔN HỌC HỆ THỐNG ROBOT THÔNG MINH**

Ngành: Công nghệ Kỹ thuật Điện tử - Truyền thông

HÀ NỘI - 2023

Mục lục

1	LÀM QUEN VỚI HỆ THỐNG THÍ NGHIỆM	1
1.1	Nền tảng mô phỏng	1
1.1.1	Giao diện VREP	3
1.1.2	Viết chương trình trong VREP	7
1.1.3	Sử dụng API kết nối Matlab và VREP	10
1.1.4	Một số câu lệnh thường dùng trong VREP	13
1.2	Nền tảng robot thực	14
1.2.1	Cấu trúc phần cứng	14
1.2.2	Lập trình giao tiếp với Robot	16
1.2.3	Hệ thống định vị trong nhà MarvelMind	19
1.3	Bài tập thực hành	25
1.3.1	Bài tập 1.1: Làm quen với môi trường mô phỏng VREP	25
1.3.2	Bài tập 1.2: Mô phỏng Robot tránh vật cản trên VREP	25
1.3.3	Bài tập 1.3: Làm quen với phần cứng robot thực	25
1.3.4	Bài tập 1.4: Thí nghiệm tránh vật cản với robot thực	26
1.3.5	Bài tập 1.5: Hệ thống định vị trong nhà MarvelMind	26
2	MÔ HÌNH ĐỘNG HỌC ROBOT 2 BÁNH VI SAI	28
2.1	Động học thuận	28
2.2	Động học ngược	30
2.3	Bài tập thực hành	32
2.3.1	Bài tập 2.1 Mô phỏng động học thuận robot	32
2.3.2	Bài tập 2.2 Mô phỏng động học ngược robot	32
2.3.3	Bài tập 2.3 Thực nghiệm động học thuận	33
2.3.4	Bài tập 2.4 Thực nghiệm động học ngược	33

3	KỸ THUẬT ODOMETRY	34
3.1	Phương pháp Odometry	34
3.2	Hướng dẫn điều khiển Robot sử dụng encoder trong Coppeliasim	37
3.3	Bài tập thực hành	37
3.3.1	Bài tập mô phỏng	37
3.3.2	Bài tập thực nghiệm	38
4	THUẬT TOÁN DẪN ĐƯỜNG BUG	40
4.1	Bug-1	40
4.2	Bug-2	42
4.3	Điều khiển hành vi	43
4.3.1	Hành vi bám tường	43
4.3.2	Hành vi đi tới đích	44
4.4	Bài tập thực hành	45
4.4.1	Mô phỏng	45
4.4.2	Thực nghiệm	46
5	KỸ THUẬT LẬP ĐƯỜNG ĐI	47
5.1	Lập đường đi cho robot	47
5.2	Thuật toán A^*	47
5.3	Thuật toán RRT	50
5.4	Bài tập thực hành	54
5.4.1	Mô phỏng	54
5.4.2	Thực nghiệm	54
A	MÔ HÌNH HAI BÁNH VI SAI	55
A.1	Mô hình hai bánh vi sai	55

Danh sách hình vẽ

1.1	Giao diện người dùng trong phần mềm VREP	2
1.2	Robot hai bánh vi sai sử dụng trong mô phỏng	3
1.3	Giao diện người dùng trong phần mềm Coppeliasim	4
1.4	Thanh công cụ thứ nhất	5
1.5	Thanh công cụ thứ hai	5
1.6	Thư viện mô hình	6
1.7	Thanh thành phần điều khiển mô hình	7
1.8	Màn hình và camera trong cảnh mô phỏng	8
1.9	So sánh các phương pháp lập trình điều khiển trong Coppeliasim	9
1.10	Cấu trúc phần mềm của CoppeliaSim	10
1.11	Câu lệnh khai báo khởi tạo ban đầu	13
1.12	Câu lệnh cài đặt vận tốc vào khớp bánh xe	13
1.13	Câu lệnh đọc cảm biến khoảng cách	14
1.14	Câu lệnh lấy thông số từ các cảm biến	14
1.15	Robot hai bánh vi sai sử dụng trong thực hành	15
1.16	Robot hai bánh vi sai được sử dụng trong các thí nghiệm	15
1.17	Sơ đồ kết nối phần cứng robot	16
1.18	Cấu trúc chương trình Arduino	17
1.19	Quá trình	17
1.20	Kiểm tra kết nối Arduino với Máy tính để nạp code	18
1.21	Biên dịch chương trình trên Arduino IDE	18
1.22	Cửa sổ thông báo trạng thái chương trình trên Arduino IDE	19
1.23	Hệ thống định vị trong nhà Marvelmind	19
1.24	Thiết lập chế độ hoạt động của beacon bằng DIP switch	20
1.25	Giao diện Dashboard	21

1.26	Bảng khoảng cách	22
1.27	Danh sách các thiết bị có trong hệ thống	22
1.28	Hai loại beacon	23
1.29	Thanh công cụ cài đặt	24
2.1	Mô hình của robot	29
2.2	Điều khiển Pose tới Pose	31
3.1	Mô hình hoạt động của Encoder	35
3.2	Mô hình mô tả Odometry	35
3.3	Mô tả cách tính Phương pháp Odometry	36
3.4	Lấy vận tốc bánh xe trong phần mềm mô phỏng Coppeliasim	37
3.5	Bài tập 1: Robot di chuyển theo đường thẳng	38
4.1	Đường đi của thuật toán Bug-1	41
4.2	Đường đi của thuật toán Bug-2	42
4.3	Hành vi bám tường	43
4.4	Hành vi đi tới đích	44
5.1	Các bước thực hiện thuật toán A^*	49
5.2	Lưu đồ thuật toán Astar	50
5.3	Trường hợp x_{new} hợp lệ	51
5.4	Trường hợp x_{new} không hợp lệ	52
5.5	Đường đi được lập bằng RRT	53
5.6	Lưu đồ thuật toán RRT	53
A.1	Mô hình hai bánh vi sai	55
A.2	Các trường hợp chuyển động của robot hai bánh vi sai	56
A.3	Động học robot hai bánh vi sai [thêm trích dẫn báo]	56

LÀM QUEN VỚI HỆ THỐNG THÍ NGHIỆM

Mục đích: Bài này giúp sinh viên làm quen với hệ thống thí nghiệm mô phỏng và thực nghiệm được sử dụng xuyên suốt trong các bài thực hành của môn học. Hệ thống sử dụng một nền tảng robot di động 2 bánh vi sai được thiết kế tương đồng về kích thước và các tham số cơ bản. Kết thúc bài học, sinh viên có thể:

- Hiểu được các thành phần cơ bản của một robot di động 2 bánh vi sai đơn giản; hiểu cách kết nối, tích hợp các thành phần hệ thống để tạo thành một robot hoàn chỉnh có thể vận hành được trên mô phỏng và thực nghiệm.
- Hiểu được cách lập trình giao tiếp cơ bản với nền tảng robot mô phỏng và robot thực; có thể tự viết các chương trình đơn giản đọc giá trị các cảm biến của robot; điều khiển các cơ cấu chuyển động thực hiện các di chuyển, tránh vật cản.

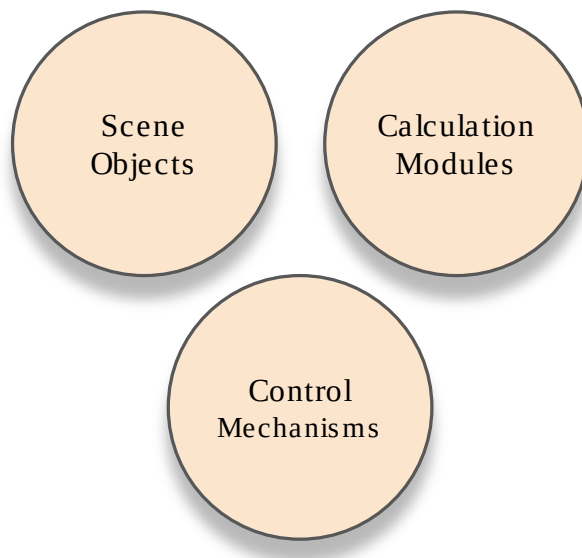
1.1 Nền tảng mô phỏng

Các bài thực hành mô phỏng trong môn học này sử dụng nền tảng thí nghiệm robot ảo VREP (Virtual Robot Experimentation Platform), tên gọi khác Coppeliasim. Đây là nền tảng thí nghiệm robot giả lập môi trường vật lý, cấu trúc phân tán. Trong VREP, các cảm biến, cơ cấu cơ khí, robot và các hệ thống tổng thể có thể được mô hình hóa và mô phỏng theo nhiều cách khác nhau, gần giống với môi trường vật lý thực. Điều này cho phép người dùng có thể tạo mẫu và đánh giá nhanh mô hình, thử nghiệm các điều khiển trước khi đi chế tạo hệ thống thật. VREP có 03 thành phần chính như mô tả trong Hình 1.1 gồm:

- Đối tượng cảnh (Scence Objects) có 14 loại đối tượng khác nhau như khớp

(Joints), hệ trục tham chiếu (Dummies/reference frames), cảm biến gần (proximity sensors)...

- Các mô-đun tính toán (Calculation modules) có 05 loại thuật toán cơ bản gồm phát hiện va chạm (collision detection), tính toán khoảng cách tối thiểu (minimum distance calculation), động học thuận/ngược (forward/inverse kinematics), lập kế hoạch đường đi/chuyển động (path/motion planning), động lực học (Physics/Dynamics).
- Các cơ chế điều khiển (Control Mechanisms) như qua chương trình nhúng, plugin, nút ROS, qua API, hoặc từ các giải pháp tự tạo với các ngôn ngữ lập trình như C/C++, Python, Java, Lua, Matlab.



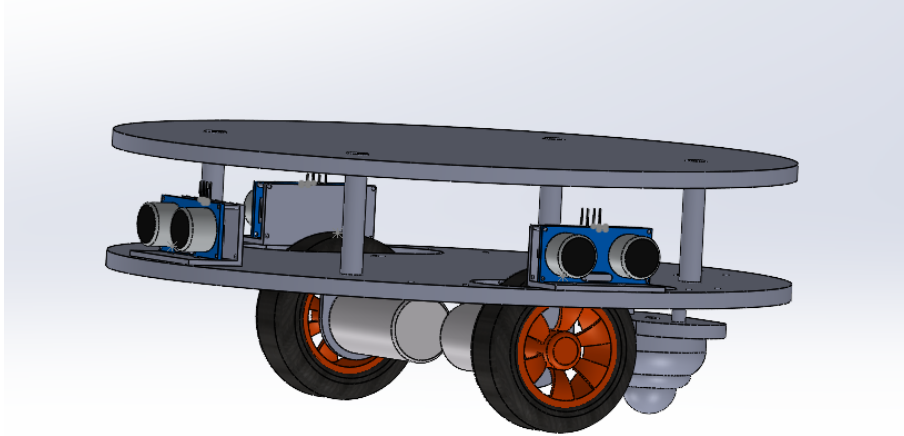
Hình 1.1: Giao diện người dùng trong phần mềm VREP

Tùy từng bài toán, người thiết kế có thể kết hợp các đối tượng khác nhau trong 03 thành phần nêu trên để tạo ra một hệ thống hoàn chỉnh. Điều này làm cho VREP rất linh hoạt và thích hợp cho các ứng dụng về Robot. Robot sử dụng trong các bài thực hành của môn học này có dạng đĩa tròn như Hình 1.2 với các thông số:

- Đường kính robot: $25cm$
- Chiều cao robot: $10cm$
- Bán kính bánh xe $R = 6cm$

- Khoảng cách giữa 2 bánh xe $L = 16.65cm$

Robot mô phỏng có 03 cảm biến đo khoảng cách; 01 cảm biến góc hướng và có khả năng tự định vị giống như sử dụng mô đun GPS.



(a) Robot Mô phỏng

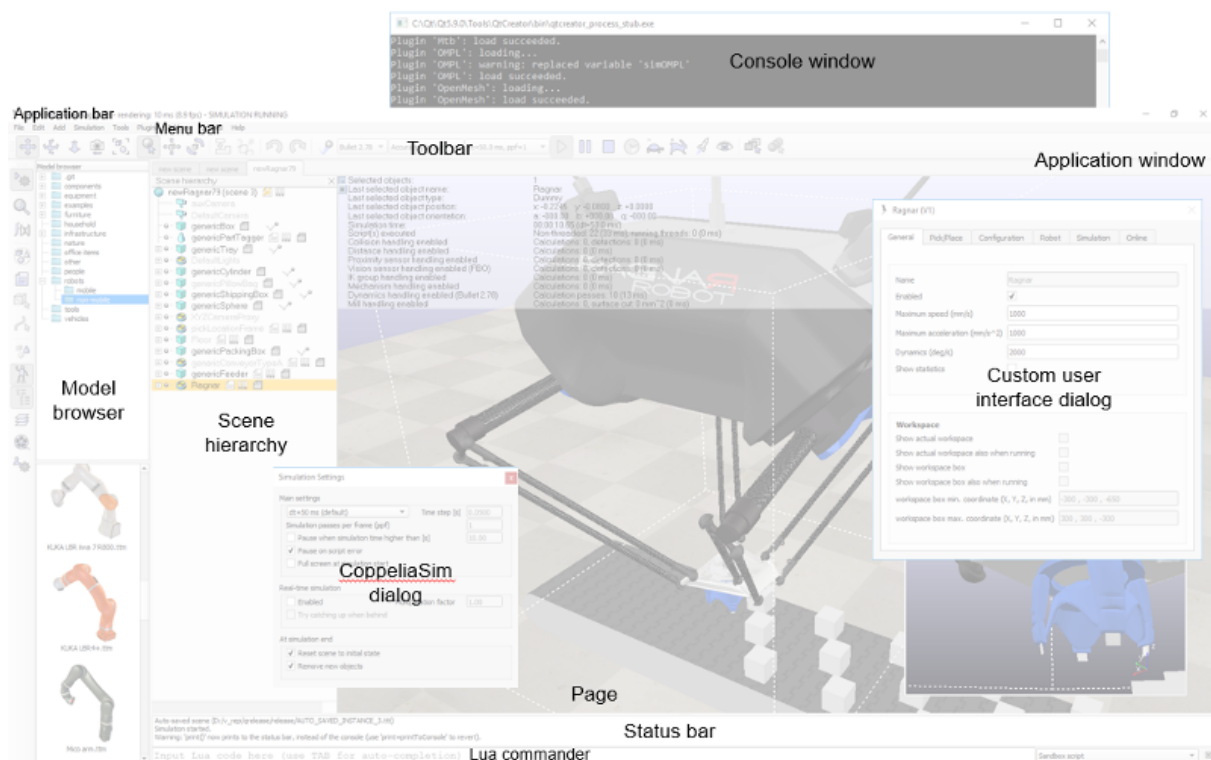
Hình 1.2: Robot hai bánh vi sai sử dụng trong mô phỏng

1.1.1 Giao diện VREP

Giao diện VREP được minh họa trong Hình 1.3, gồm các thành phần chính như sau:

- **Một cửa sổ giao diện điều khiển** (console window): Khi ứng dụng VREP bắt đầu, một cửa sổ điều khiển được tạo nhưng lại ẩn trực tiếp. Cửa sổ mặc định này để ẩn cửa sổ bảng điều khiển có thể được thay đổi trong hộp thoại Cài đặt người dùng.
- **Một cửa sổ ứng dụng** (application window): Cửa sổ ứng dụng là cửa sổ chính của ứng dụng. Nó được sử dụng để hiển thị, chỉnh sửa, mô phỏng và tương tác với một kịch bản. Các nút chuột trái và phải, con lăn chuột cũng như bàn phím có các chức năng cụ thể khi được kích hoạt trong cửa sổ ứng dụng. Trong cửa sổ ứng dụng, các chức năng của thiết bị đầu vào (chuột và bàn phím) có thể khác nhau tùy theo ngữ cảnh hoặc vị trí kích hoạt.
- **Một số hộp thoại** (several dialogs): Bên cạnh cửa sổ ứng dụng, người dùng cũng có thể chỉnh sửa và tương tác với kịch bản bằng cách điều chỉnh cài đặt

hộp thoại hoặc tham số. Mỗi hộp thoại nhóm một tập hợp các hàm liên quan hoặc các hàm áp dụng cho cùng một đối tượng mục tiêu. Nội dung của hộp thoại có thể nhạy cảm với ngữ cảnh (ví dụ: phụ thuộc vào trạng thái lựa chọn đối tượng).



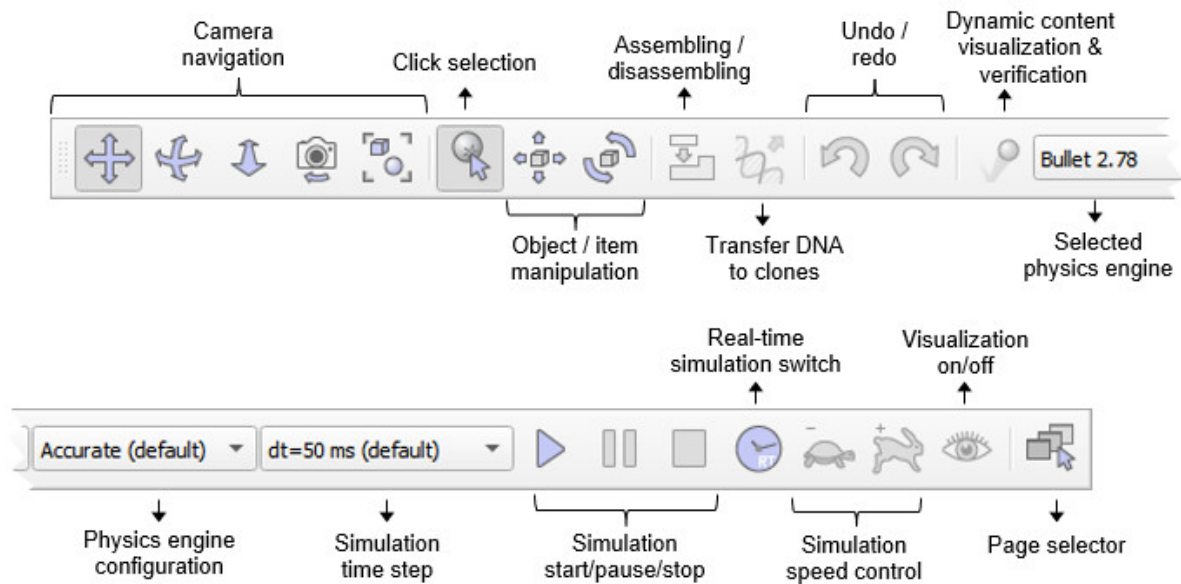
Hình 1.3: Giao diện người dùng trong phần mềm Coppeliasim

Khi khởi chạy VREP, VREP sẽ khởi chạy một kịch bản mặc định. Người dùng có thể tự do mở một số kịch bản song song. Mỗi kịch bản chia sẻ cửa sổ ứng dụng và hộp thoại với các kịch bản khác, nhưng chỉ nội dung kịch bản hoạt động mới hiển thị trong cửa sổ ứng dụng hoặc hộp thoại (chỉ một kịch bản hiển thị tại một thời điểm nhất định).

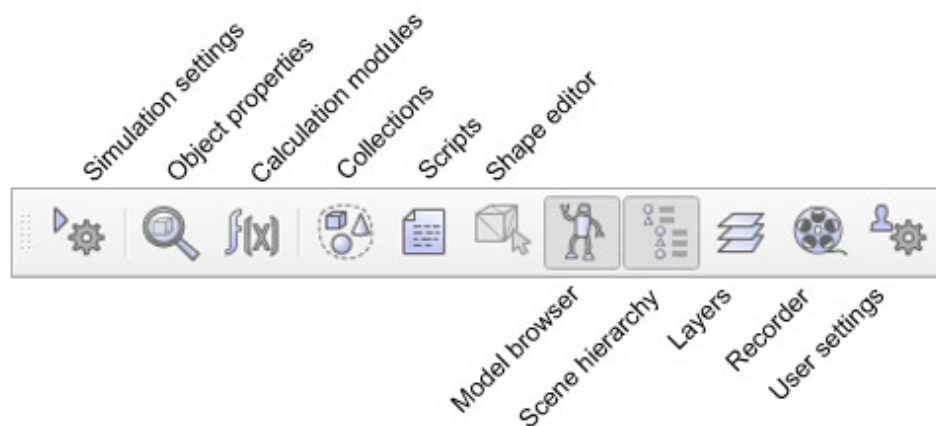
Các thanh công cụ trong mỗi kịch bản có các chức năng riêng biệt như Hình 1.4 và Hình 1.5.

- **Thanh ứng dụng** (Application bar): Thanh ứng dụng cho biết tên tệp của cảnh hiện đang được hiển thị, thời gian được sử dụng cho một lượt kết xuất (một lượt hiển thị) và trạng thái hiện tại của trình giả lập (trạng thái hoặc loại mô phỏng của chế độ chỉnh sửa đang hoạt động).
- **Thanh menu** (Menu bar): thanh menu cho phép truy cập hầu hết các chức năng của trình giả lập.

- **Thanh công cụ** (Toolbar - Hình 1.4, 1.5): thanh công cụ trình bày các chức năng thường được truy cập (ví dụ: thay đổi chế độ điều hướng, chọn trang khác, v.v.).

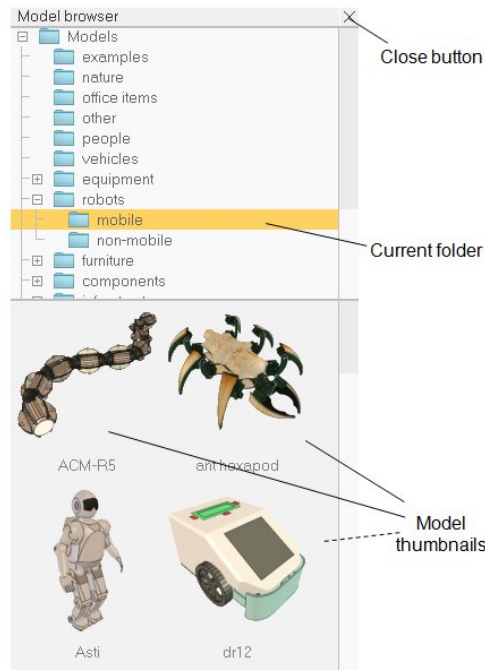


Hình 1.4: Thanh công cụ thứ nhất



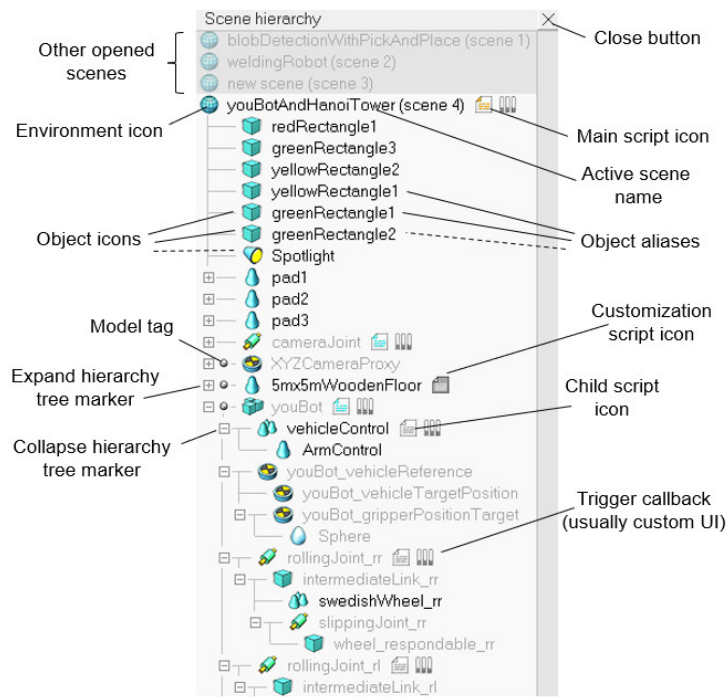
Hình 1.5: Thanh công cụ thứ hai

- **Thư viện mô hình** (Model browser - Hình 1.6): thư viện mô hình được hiển thị theo mặc định, nhưng có thể được bật bằng nút thanh công cụ tương ứng. Nó hiển thị ở phần trên cấu trúc thư mục mô hình CoppeliaSim và ở phần dưới, hình thu nhỏ của các mô hình có trong thư mục đã chọn. Hình thu nhỏ có thể được kéo và thả vào cảnh để tự động tải mô hình liên quan. Hình thu nhỏ được bật có vẻ tối nếu khu vực thả không được hỗ trợ hoặc không phù hợp.



Hình 1.6: Thư viện mô hình

- **Thanh mô hình** (Scene hierarchy - Hình 1.7): thanh mô hình được hiển thị theo mặc định, nhưng có thể được chuyển đổi bằng nút thanh công cụ tương ứng. Nó hiển thị nội dung của một cảnh (nghĩa là tất cả các đối tượng cảnh tạo thành một cảnh). Do các đối tượng cảnh được xây dựng theo cấu trúc giống như cấu trúc phân cấp nên cấu trúc phân cấp cảnh hiển thị một cây của cấu trúc phân cấp này và các phần tử riêng lẻ có thể được mở rộng hoặc thu gọn. Nhấp đúp vào biểu tượng sẽ mở/đóng hộp thoại thuộc tính liên quan đến biểu tượng được nhấp. Nhấp đúp vào một bí danh đối tượng cho phép chỉnh sửa nó. Con lăn chuột cũng như thao tác kéo thanh cuộn của chế độ xem phân cấp cảnh cho phép chuyển nội dung lên/xuống hoặc sang trái/phải. Lựa chọn điều khiển và thay đổi luôn được hỗ trợ. Các đối tượng trong phân cấp cảnh có thể được kéo và thả vào một đối tượng khác để tạo mối quan hệ cha-con. Hệ thống phân cấp cảnh sẽ hiển thị một nội dung khác nếu trình mô phỏng ở trạng thái chế độ chỉnh sửa.
- **Trang** (Page - Hình 1.8): Một trang trong Coppeliasim là bề mặt xem chính của một cảnh. Nó không trực tiếp là một chế độ xem nhưng có thể chứa một, hai hoặc nhiều màn hình xem nếu cần. Một màn hình xem (View) là những gì được sử dụng để hiển thị nội dung hình ảnh của một đối tượng cụ thể phải là một đối tượng có thể xem được.

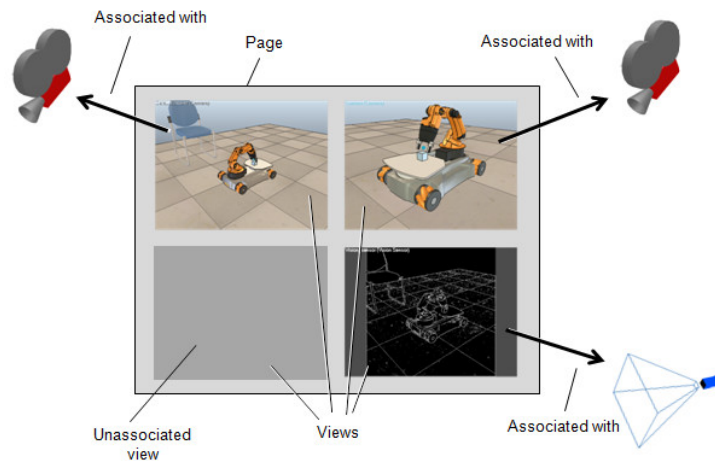


Hình 1.7: Thanh thành phần điều khiển mô hình

- **Thanh trạng thái** (Status bar): thanh trạng thái hiển thị thông tin liên quan đến các hoạt động, lệnh đã thực hiện và cũng hiển thị thông báo lỗi từ trình thông dịch tập lệnh. Từ bên trong tập lệnh, người dùng cũng có thể xuất chuỗi tới thanh trạng thái hoặc bảng điều khiển bằng hàm `sim.addLog`. Theo mặc định, thanh trạng thái chỉ hiển thị hai dòng nhưng có thể thay đổi kích thước bằng cách kéo dẫn thanh.
- **Vòng lệnh LUA** (Lua commander): vòng lặp đọc-đánh giá-in, bổ sung đầu vào văn bản vào thanh trạng thái Coppeliasim, cho phép nhập và thực thi mã Lua một cách nhanh chóng, giống như trong một thiết bị đầu cuối.

1.1.2 Viết chương trình trong VREP

Coppeliasim là một trình mô phỏng có khả năng tùy biến cao: mọi khía cạnh của mô phỏng đều có thể được tùy chỉnh. Hơn nữa, bản thân trình giả lập có thể được tùy chỉnh và điều chỉnh để hoạt động chính xác như mong muốn. Điều này được cho phép thông qua Giao diện lập trình ứng dụng (API) phức tạp. Hơn 6 phương pháp lập trình hoặc viết mã khác nhau được hỗ trợ, mỗi phương pháp có những ưu điểm riêng (và rõ ràng là cả nhược điểm) so với những phương pháp khác, nhưng cả sáu phương pháp đều tương thích lẫn nhau (tức là có thể được sử dụng cùng nhau).



Hình 1.8: Màn hình và camera trong cảnh mô phỏng

dụng cùng một lúc hoặc thậm chí là song hành). Thực thể điều khiển của mô hình, cảnh hoặc bản thân trình mô phỏng có thể được đặt bên trong:

- **Tập lệnh nhúng** (nghĩa là tùy chỉnh mô phỏng (tức là cảnh hoặc mô hình) thông qua tập lệnh): phương pháp này, bao gồm việc viết tập lệnh Lua hoặc Python, rất dễ dàng và linh hoạt, với khả năng tương thích được đảm bảo với mọi cài đặt CoppeliaSim mặc định khác (miễn là các chức năng API tùy chỉnh không được sử dụng hoặc được sử dụng với các plugin phân tán). Phương pháp này cho phép tùy chỉnh một mô phỏng cụ thể, một cảnh mô phỏng và ở một mức độ nhất định chính trình giả lập đó. Đây là phương pháp lập trình dễ nhất và được sử dụng nhiều nhất.
- **một tiện ích bổ sung hoặc tập lệnh hộp cát:** phương pháp này, bao gồm việc viết các tập lệnh Lua hoặc Python, cho phép nhanh chóng tùy chỉnh chính trình mô phỏng. Tiện ích bổ sung (hoặc tập lệnh hộp cát) có thể tự động bắt đầu và chạy trong nền hoặc chúng có thể được gọi là hàm (ví dụ: thuận tiện khi viết trình nhập/xuất). Các tiện ích bổ sung không nên dành riêng cho một mô phỏng hoặc mô hình nhất định, chúng nên cung cấp một chức năng giới hạn giả lập, chung chung hơn.
- **plugin** (nghĩa là tùy chỉnh trình giả lập và/hoặc mô phỏng thông qua plugin): phương pháp này về cơ bản bao gồm việc viết plugin cho CoppeliaSim. Thông thường, plugin chỉ được sử dụng để cung cấp mô phỏng với các lệnh API tùy chỉnh và do đó được sử dụng cùng với phương pháp đầu tiên. Những lần khác, plugin được sử dụng để cung cấp cho CoppeliaSim một chức năng đặc biệt yêu cầu khả năng tính toán nhanh (phần lớn thời gian các tập lệnh chậm hơn

	Embedded script	Add-on / sandbox script	Plugin	Remote API client	ROS / ROS2 node	ZeroMQ node
Control entity is external (i.e. can be located on a robot, different machine, etc.)	No	No	No	Yes	Yes	Yes
Difficulty to implement	Easiest	Easiest	Relatively easy	Easy	Relatively easy	Easy
Supported programming language	Lua, Python	Lua, Python	C/C++	C/C++, Python, Java, JavaScript, Matlab, Octave	Any ¹	Any
Code execution speed	Relativ. fast ²	Relativ. fast ²	Fast	Depends on programming language	Depends on programming language	Depends on programming language
Communication lag	None ³	None ³	None	Yes	Yes	Yes
Control entity can be fully contained in a scene or model, and is highly portable	Yes	No	No	No	No	No
Control entity relies on	CoppeliaSim	CoppeliaSim	CoppeliaSim	Sockets, ZeroMQ or WebSockets	ROS / ROS2 framework	ZeroMQ
Stepped operation ⁴	Yes, inherent	Yes, inherent	Yes, inherent	Yes	Yes	Yes
Non-stepped operation ⁴	Yes, via threads	Yes, via threads	No (threads available, but API access forbidden)	Yes	Yes	Yes

¹⁾ Depends on ROS / ROS2 bindings

²⁾ Depends on the programming language, but the execution of API functions is very fast

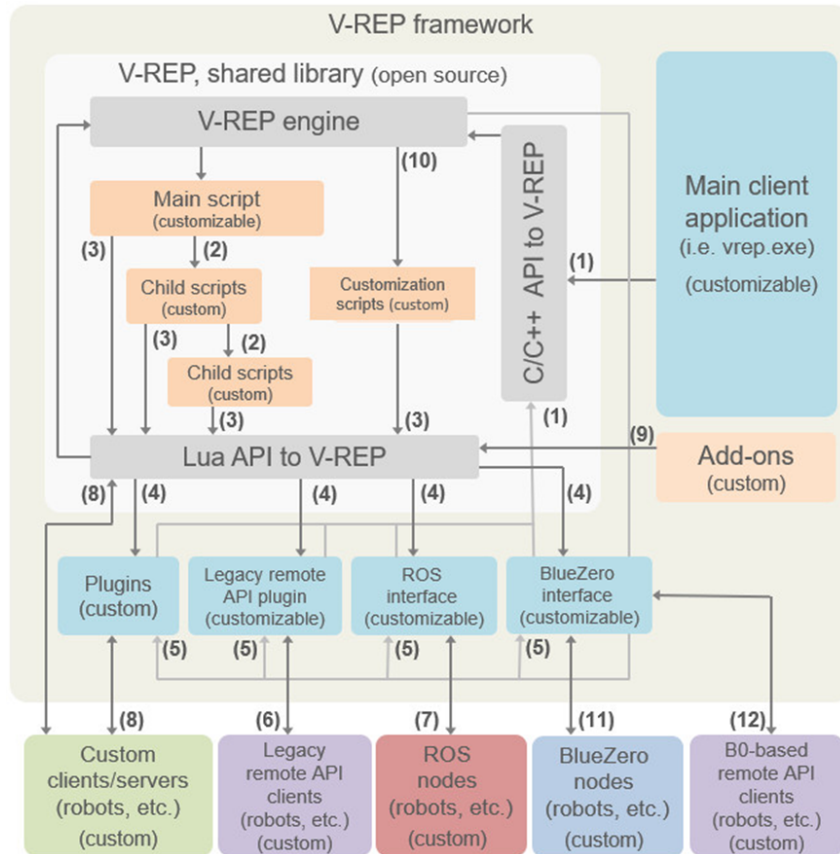
³⁾ Lua scripts are executed in CoppeliaSim's main thread, Python scripts are executed in separate processes

⁴⁾ Stepped as in *synchronized* with each simulation step

Hình 1.9: So sánh các phương pháp lập trình điều khiển trong Coppeliasim

so với các ngôn ngữ được biên dịch), một giao diện cụ thể cho một thiết bị phần cứng (ví dụ: rô-bốt thực) hoặc một giao diện truyền thông đặc biệt với thế giới bên ngoài.

- **ứng dụng khách API từ xa** (nghĩa là tùy chỉnh trình giả lập và/hoặc mô phỏng thông qua ứng dụng ứng dụng khách API từ xa): phương pháp này cho phép một ứng dụng bên ngoài (ví dụ: nằm trên rô-bốt, máy khác, v.v.) kết nối với CoppeliaSim một cách rất dễ dàng, sử dụng các lệnh API từ xa.
- **nút ROS** (nghĩa là tùy chỉnh trình mô phỏng và/hoặc mô phỏng qua nút ROS): phương pháp này cho phép ứng dụng bên ngoài (ví dụ: nằm trên rô-bốt, máy khác, v.v.) kết nối với CoppeliaSim thông qua ROS, Hệ điều hành rô-bốt.
- **một nút giao tiếp TCP/IP, ZeroMQ, v.v.:** phương pháp này cho phép một ứng dụng bên ngoài (ví dụ: nằm trên rô-bốt, máy khác, v.v.) kết nối với CoppeliaSim thông qua các phương tiện giao tiếp khác nhau.



Hình 1.10: Cấu trúc phần mềm của CoppeliaSim

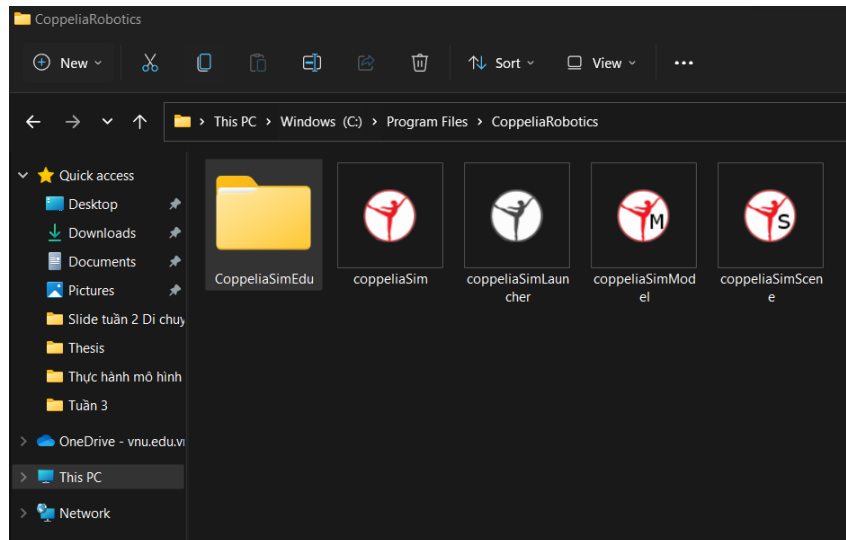
1.1.3 Sử dụng API kết nối Matlab và VREP

Kiến trúc phần mềm của v-rep được hiển thị bên dưới hình 1.10. Nó tương tác với công cụ tính toán mô phỏng thông qua môi trường lập trình Lua tích hợp bên trong và sau đó hoàn thành chức năng điều khiển của mô hình robot ảo. Đồng thời, giao diện API từ xa tương tác với API Lua bên trong, cho phép các chương trình bên ngoài điều khiển mô hình robot ảo của CoppeliaSim thông qua API từ xa, giao diện này tương tác với phần mềm khác thông qua Socket.

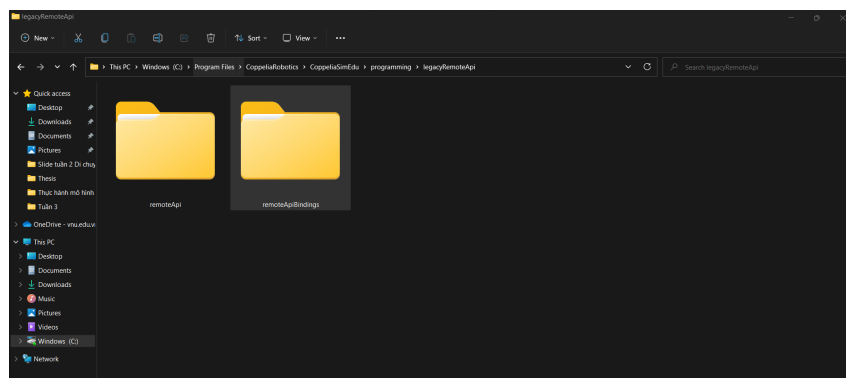
Hướng dẫn các bước kết nối môi trường CoppeliaSim và Matlab:

Bước 1: Tìm kiếm các tệp liên quan đến liên kết API giữa Coppeliasim và Matlab.

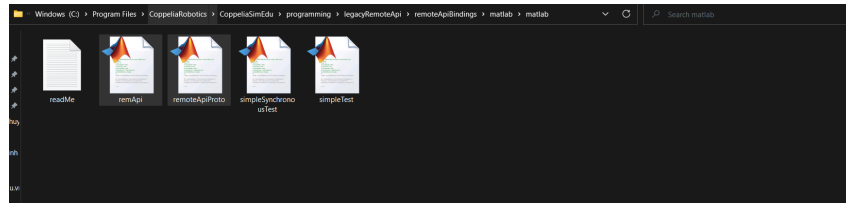
Để tìm ba tệp mà kết nối giữa CoppeliaSim và Matlab phụ thuộc, cần thực hiện các bước sau: Đi đến tệp tải xuống của CoppeliaSim (Có thể ở một trong hai đường dẫn file C:\Program Files\CoppeliaRobotics hoặc file C:\Program Files(x86)\CoppeliaRobotics) và tiếp tục chọn file **CoppeliaSimEdu**.



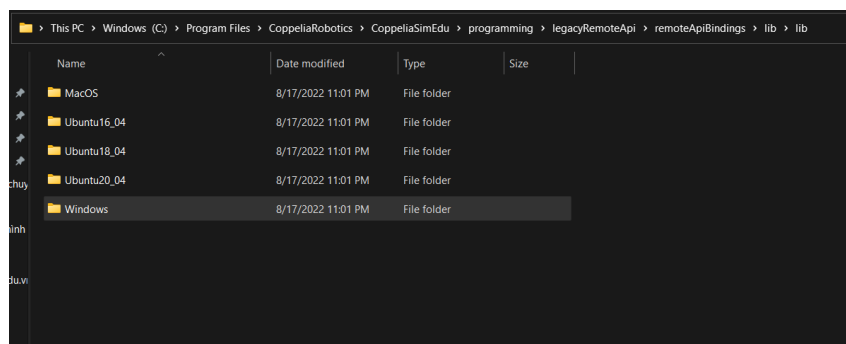
Sau khi truy cập vào thư mục **CoppeliasimEdu**, tiếp tục tìm thư mục **remoteApiBindings**. Có thể sử dụng **sreach** trên thanh công cụ để tìm kiếm với từ khóa **remoteApiBindings**. Hoặc truy cập theo đường dẫn
`..\CoppeliasimRobotics\CoppeliasimEdu\programming\legacyRemoteApi`



Truy cập vào thư mục **remoteApiBindings**. Ở đây sẽ có 5 ngôn ngữ tương ứng được sử dụng: Matlab, Java, Lua, Python, Octave. Chọn matlab và copy 2 file liên kết matlab và Coppeliasim này. **remApi** và **remoteApiProto** như hình miêu tả dưới.



Ngoài ra, đang trong thư mục **remoteApiBindings** còn phải truy cập theo đường dẫn để chọn hệ điều hành kết nối tương ứng theo đường dẫn `../remoteApiBindings/lib/lib`



Ví dụ khi sử dụng window, cần tìm ra file **remoteApi.dll** để copy và là file liên kết cần có để liên kết giữa Matlab và CoppeliaSim.

Chú ý quan trọng, khi không sử dụng được đường dẫn để tìm kiếm hoặc không tìm thấy file, hãy nghĩ tới việc sử dụng công cụ tìm kiếm trên thanh công cụ để tìm ra 3 file quan trọng với 3 từ khóa tìm kiếm: **remoteApi.dll**, **remApi.m** và **remoteApiProto.m**.

Ba tệp tin **remoteApi.dll**, **remApi.m** và **remoteApiProto.m** là các tệp chính cho đồng mô phỏng giữa CoppeliaSim và Matlab. Khi thực hiện mô phỏng, ba tệp này phải đặt ở trong cùng thư mục với thư mục mô hình trong CoppeliaSim và mã Matlab tương ứng với nội dung mô phỏng.

Trong thư mục mô phỏng, chương trình chính được để trong file **main.m**. Các hướng dẫn về cấu trúc chương trình để khởi tạo kết nối với trình mô phỏng được chú thích rõ ràng trong file **main.m**.

1.1.4 Một số câu lệnh thường dùng trong VREP

Khởi tạo ban đầu: Sử dụng câu lệnh để khai báo các thành phần sẽ sử dụng để lấy dữ liệu trong robot.

simxGetObjectHandle	
Description	Retrieves an object handle based on its path and alias.
Matlab synopsis	[number returnCode,number handle]=simxGetObjectHandle(number clientID,string objectPath,number operationMode)
Matlab parameters	clientID: the client ID. refer to simxStart . objectPath: the path of the object. See the section on accessing scene objects for details. operationMode: a remote API function operation mode . Recommended operation mode for this function is <code>simx_opmode_blocking</code>
Matlab return values	returnCode: a remote API function return code handle: the handle
Other languages	C/C++ , Python , Java , Octave

Hình 1.11: Câu lệnh khai báo khởi tạo ban đầu

Cài đặt vận tốc: Sử dụng vận tốc với câu lệnh để truyền xuống hai khớp bánh xe trong phần mềm mô phỏng. Vận tốc được đưa vào sẽ tính theo đơn vị rad/s.

simxSetJointTargetVelocity	
Description	Sets the intrinsic target velocity of a non-spherical joint. This command makes only sense when the joint mode is in torque/force mode: the dynamics functionality and the joint motor have to be enabled (position control should however be disabled)
Matlab synopsis	[number returnCode]=simxSetJointTargetVelocity(number clientID,number jointHandle,number targetVelocity,number operationMode)
Matlab parameters	clientID: the client ID. refer to simxStart . jointHandle: handle of the joint targetVelocity: target velocity of the joint (linear or angular velocity depending on the joint-type) operationMode: a remote API function operation mode . Recommended operation modes for this function are <code>simx_opmode_oneshot</code> or <code>simx_opmode_streaming</code>
Matlab return values	returnCode: a remote API function return code
Other languages	C/C++ , Python , Java , Octave

Hình 1.12: Câu lệnh cài đặt vận tốc vào khớp bánh xe

Đọc cảm biến khoảng cách: Sử dụng câu lệnh để biết khoảng cách đến vật cản và có vật cản trong môi trường theo câu lệnh:

simxReadProximitySensor	
Description	Reads the state of a proximity sensor. This function doesn't perform detection, it merely reads the result from a previous call to sim.handleProximitySensor (<code>sim.handleProximitySensor</code> is called in the default main script). See also simxGetObjectGroupData .
Matlab synopsis	[number returnCode,bool detectionState,array detectedPoint,number detectedObjectHandle,array detectedSurfaceNormalVector]= <code>simxReadProximitySensor</code> (number clientID,number sensorHandle,number operationMode)
Matlab parameters	clientID : the client ID. refer to simxStart . sensorHandle : handle of the proximity sensor operationMode : a remote API function operation mode . Recommended operation modes for this function are <code>simx_opmode_streaming</code> (the first call) and <code>simx_opmode_buffer</code> (the following calls)
Matlab return values	returnCode : a remote API function return code detectionState : the detection state (false=no detection) detectedPoint : the detected point coordinates (relative to the sensor reference frame) detectedObjectHandle : the handle of the detected object detectedSurfaceNormalVector : the normal vector (normalized) of the detected surface. Relative to the sensor reference frame
Other languages	C/C++ , Python , Java , Octave

Hình 1.13: Câu lệnh đọc cảm biến khoảng cách

Câu lệnh đọc cảm biến khác: Sử dụng để nhận tín hiệu từ các cảm biến GPS, cảm biến vận tốc và gia tốc:

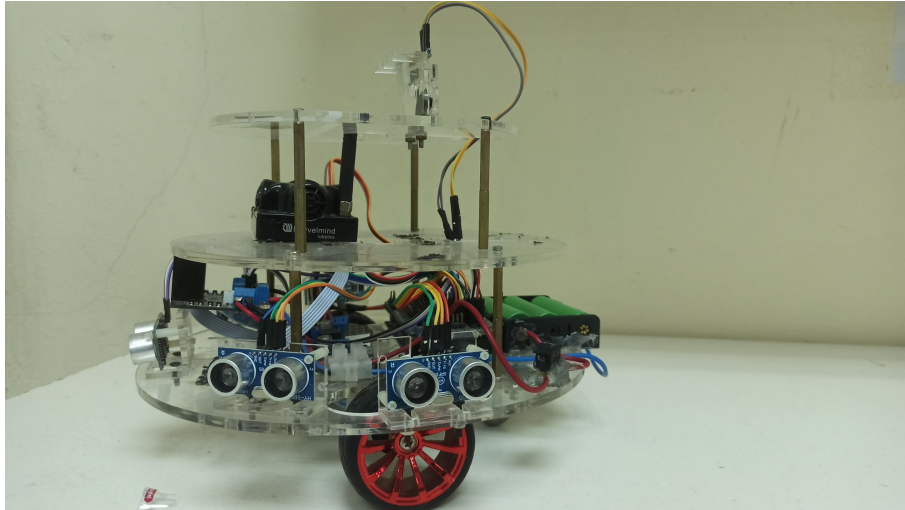
simxGetFloatSignal	
Description	Gets the value of a float signal. Signals are cleared at simulation start. See also simxSetFloatSignal , simxClearFloatSignal , simxGetInt32Signal and simxGetStringSignal .
Matlab synopsis	[number returnCode,number signalValue]= <code>simxGetFloatSignal</code> (number clientID,string signalName,number operationMode)
Matlab parameters	clientID : the client ID. refer to simxStart . signalName : name of the signal operationMode : a remote API function operation mode . Recommended operation modes for this function are <code>simx_opmode_streaming</code> (the first call) and <code>simx_opmode_buffer</code> (the following calls)
Matlab return values	returnCode : a remote API function return code signalValue : the value of the signal
Other languages	C/C++ , Python , Java , Octave

Hình 1.14: Câu lệnh lấy thông số từ các cảm biến

1.2 Nền tảng robot thực

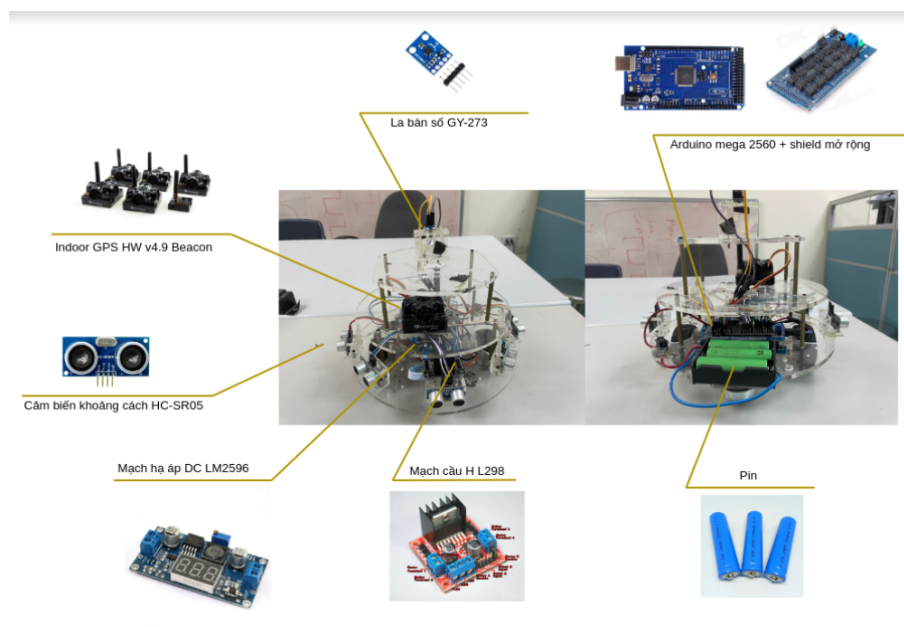
1.2.1 Cấu trúc phần cứng

Hình 1.15 là ảnh chụp robot hai bánh vi sai được dùng trong thực nghiệm cho môn học. Robot có kích thước tương đồng với robot mô phỏng.



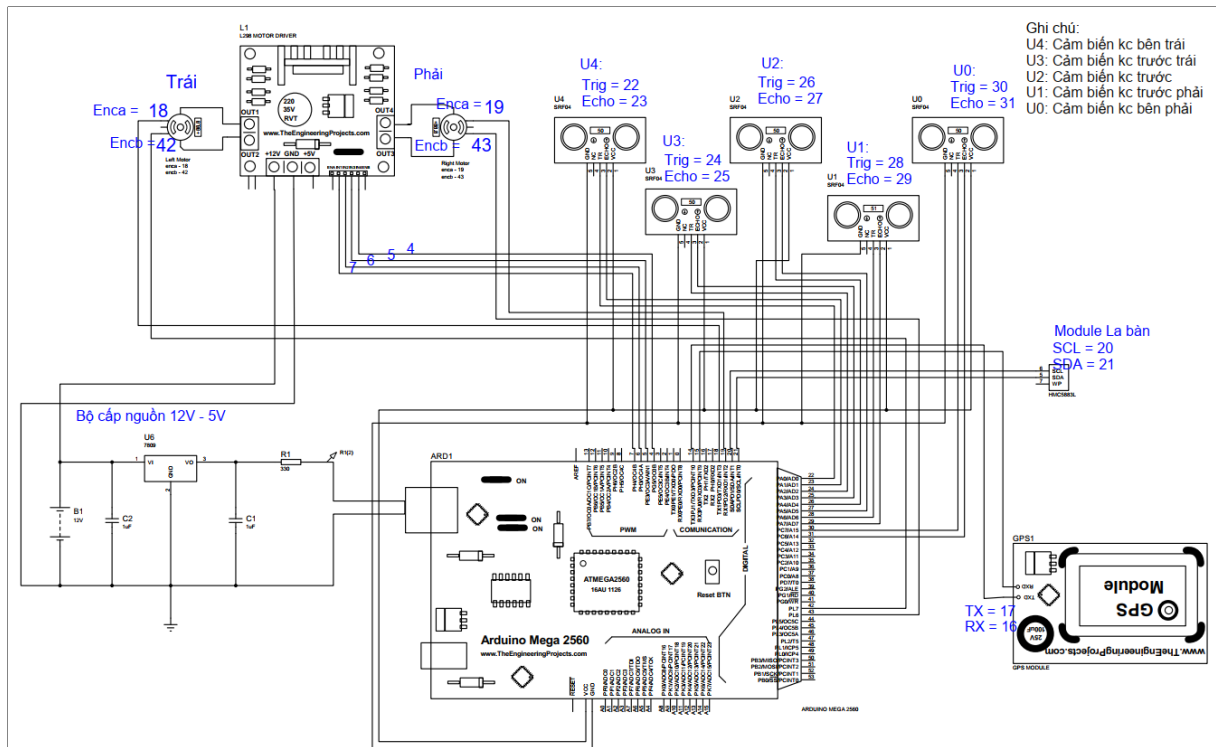
Hình 1.15: Robot hai bánh vi sai sử dụng trong thực hành

Cấu trúc phần cứng của robot được mô tả trong Hình 1.16. Nó có thể gồm 3 tầng được ghép từ các tấm nhựa trong suốt hình tròn: tầng 1 chứa nguồn nuôi, bộ điều khiển trung tâm sử dụng Arduino mega 2560, mạch điều khiển động cơ L298, 05 cảm biến siêu âm SRF05; tầng 2 đặt Beacon định vị; tầng 3 đặt cảm biến la bàn số. Robot sử dụng động cơ DC GA5 hoạt động trong dải điện áp từ 3-13 VDC (trong thiết kế này sử dụng điện áp 12V), có đĩa lập mã 374 xung, 2 kênh.



Hình 1.16: Robot hai bánh vi sai được sử dụng trong các thí nghiệm

Sơ đồ đấu dây của robot chỉ ra trong Hình 1.17, cụ thể như sau:



Hình 1.17: Sơ đồ kết nối phần cứng robot

1.2.2 Lập trình giao tiếp với Robot

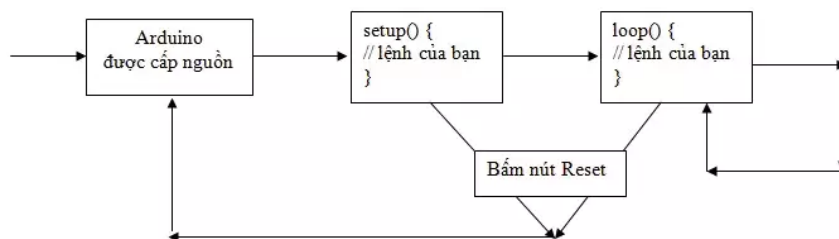
Cấu trúc chương trình Arduino

Mở môi trường phát triển Arduino IDE sau khi đã download và cài đặt. Cấu trúc ban đầu của chương trình Arduino IDE khá đơn giản như trong Hình 1.18, chỉ bao gồm hai hàm *setup()* và *loop()*. Khi chương trình bắt đầu chạy, những lệnh trong *setup()* sẽ được xử lý đầu tiên, ta thường dùng hàm này để khởi tạo trạng thái và giá trị của các biến hay các thông số trong chương trình.



Hình 1.18: Cấu trúc chương trình Arduino

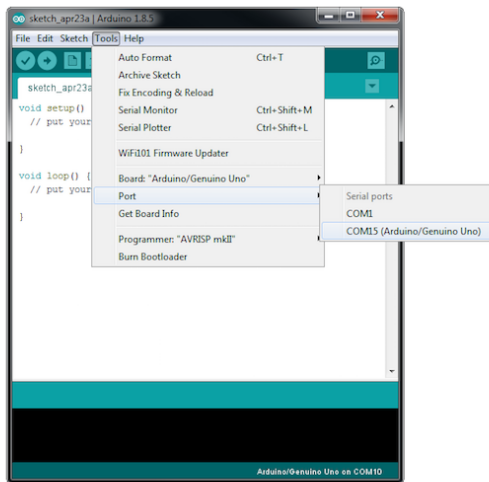
Sau khi *setup()* chạy xong, những lệnh trong *loop()* được chạy. Đây là một vòng lặp vô tận, do đó các dòng code trong hàm này sẽ được lặp đi lặp lại liên tục cho tới khi nào bạn ngắt nguồn của board Arduino mới thôi hoặc can thiệp bằng nút Reset trên bảng mạch, chương trình của bạn sẽ trở về lại trạng thái như khi Arduino mới được cấp nguồn, tức là bắt đầu chạy lại từ hàm *setup()*. Quá trình được thể hiện như trong Hình 1.19:



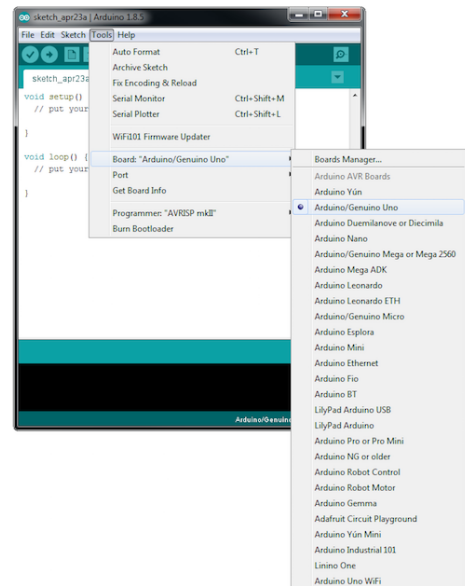
Hình 1.19: Quá trình

Lập trình và nạp code

Trước khi nạp code ta cần kiểm tra xem IDE đã kết nối tới mạch hay chưa như Hình 1.20, để làm điều này ta vào *Tools > Port* và xem cổng COM có đúng Arduino hay không. Sau khi kiểm tra đúng cổng, ta cần báo với IDE loại board Arduino nào ta đang dùng, để làm điều này ta vào *Tools > Board* và chọn “Arduino / Genuino Uno”.



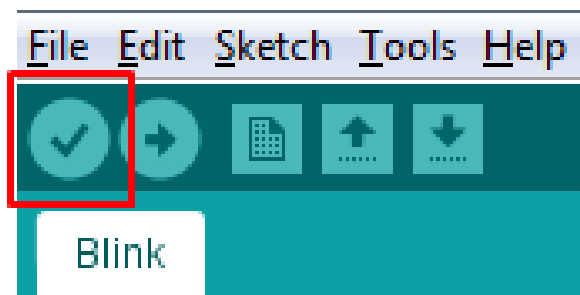
(a)



(b)

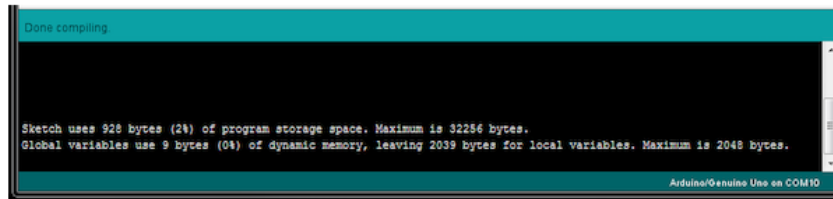
Hình 1.20: Kiểm tra kết nối Arduino với Máy tính để nạp code

Để biên dịch mã, hãy nhấp vào nút “Verify” được hiển thị bên dưới:



Hình 1.21: Biên dịch chương trình trên Arduino IDE

Khi IDE đã hoàn tất việc biên dịch, bạn sẽ thấy một kết quả ở cửa sổ đầu ra ở cuối IDE. Cửa sổ đầu ra rất hữu ích để xem các thông tin trả về thành công, lỗi, cảnh báo và việc sử dụng bộ nhớ như hình 1.22:



Hình 1.22: Cửa sổ thông báo trạng thái chương trình trên Arduino IDE

Bước cuối cùng là nạp code, để thực hiện việc này, hãy nhấp vào nút "Upload", là mũi tên ở bên phải của nút "Verify". Bao giờ xuất hiện "Done compliting" như hình 1.22 thì quá trình nạp code từ máy tính vào arduino thành công.

1.2.3 Hệ thống định vị trong nhà MarvelMind

Hệ thống định vị trong nhà dùng trong các thí nghiệm của môn học này của hàng Marvelmind như Hình 1.23 sử dụng công nghệ định vị dựa trên việc ước lượng thời gian truyền sóng siêu âm giữa beacon di động và cố định. Nó có độ chính xác lên đến $\pm 2\text{cm}$, có thể sử dụng để định vị, dẫn đường cho các robot di động, phương tiện AGV cũng như theo dõi các vật thể chuyển động. Hệ thống gồm các thành phần cơ bản sau:



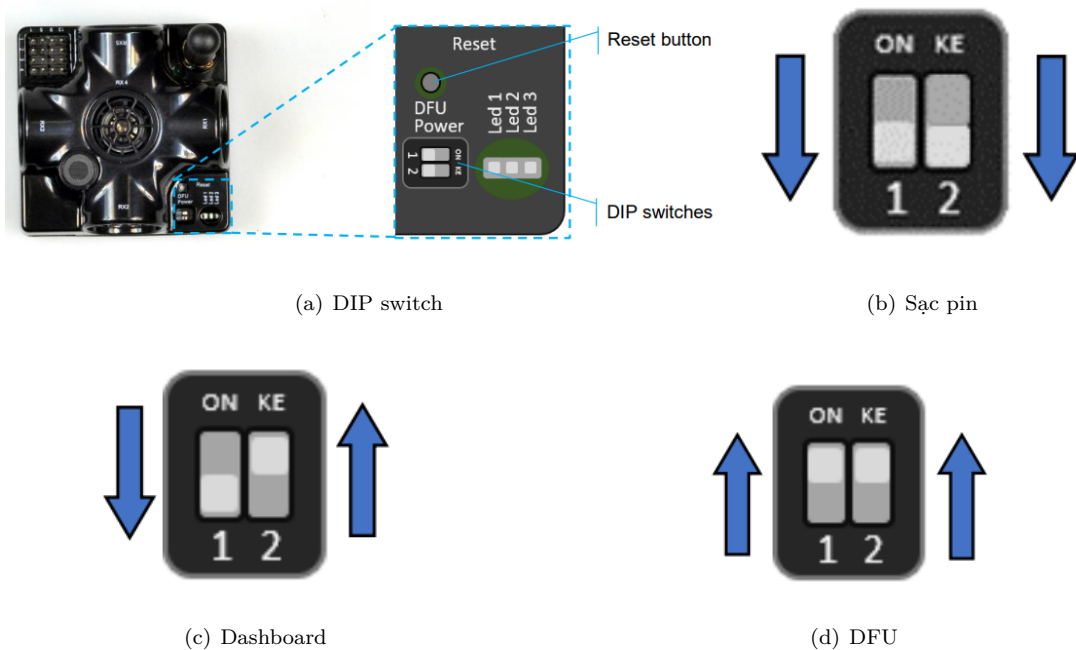
Hình 1.23: Hệ thống định vị trong nhà Marvelmind

- 04 Beacon được gắn cố định xung quanh không gian thí nghiệm.
- 01 Beacon được gắn trên robot di động.
- 01 Modem v5.1 thực hiện vai trò giao tiếp giữa các Beacon với máy tính.

Modem có thể hỗ trợ kết nối lên đến 250 Beacon, cho phép bao phủ một không gian rộng lớn và nhiều đối tượng giám sát.

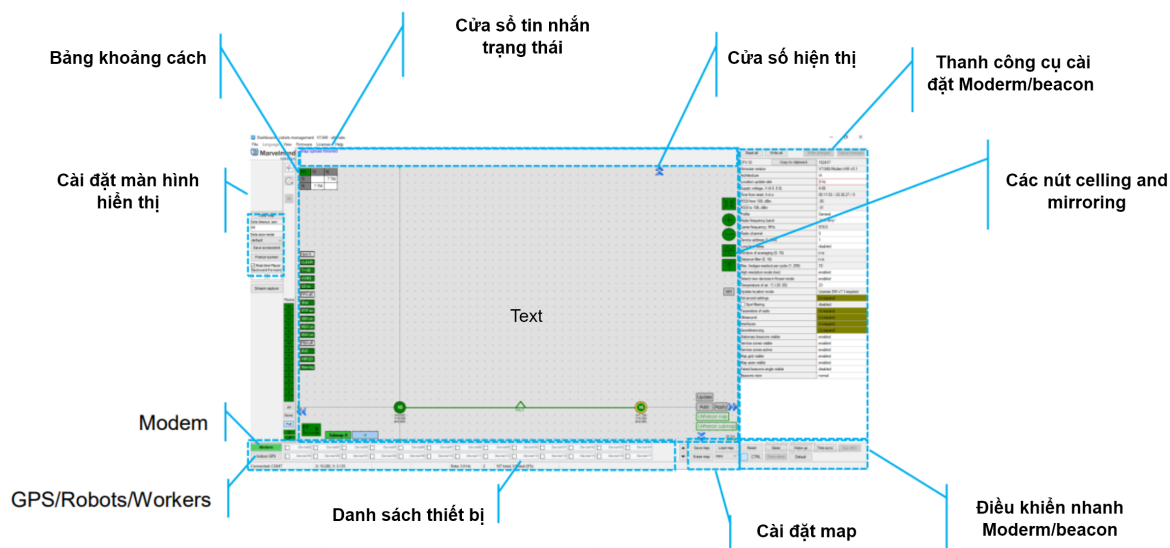
Chế độ hoạt động của các Beacon được thiết lập thông qua hai công tắc nhỏ (DIP switch) trên thân của beacon như minh họa trong Hình 1.24, gồm:

- Chế độ sạc pin.
- Chế độ làm việc trên chương trình Dashboard.
- Chế độ DFU (Trong môn học này, không cần quan tâm đến chế độ làm việc này).



Hình 1.24: Thiết lập chế độ hoạt động của beacon bằng DIP switch

Giao diện chương trình dashboard được biểu diễn trong Hình 1.25 với các thành phần chính sau đây:



Hình 1.25: Giao diện Dashboard

- Cửa sổ tin nhắn trạng thái: Nếu trường trống, nghĩa là hệ thống không tìm thấy bất kỳ sự cố nào khi khởi chạy. Nếu phát sinh bất kỳ tin nhắn nào, hãy kiểm tra lại hệ thống và khắc phục lại lỗi. Có 4 màu lỗi xuất hiện:
 - Cam – Vấn đề nhỏ
 - Màu đỏ – Vấn đề lớn
 - Màu xanh lá cây - Hoàn thành cài đặt thành công.
 - Màu xanh lam – Hoàn tất quá trình thành công
- Bảng khoảng cách: Bảng khoảng cách hiển thị khoảng cách đo được giữa tất cả các beacons như minh họa trong Hình 1.26. Bản đồ và hình ảnh hiển thị các beacons phụ thuộc nhiều vào khoảng cách, đây là một phần rất quan trọng của hệ thống. Có hai cách đo lường: 1) Đo bằng sóng siêu âm (đo tự động) và 2) đo bởi người dùng (thủ công). Có 4 màu lỗi xuất hiện:

HIDE	6	22	66	77
6		8.000	27.378	5.054
22	8.000		28.688	18.739
66	33.772	29.794		18.741
77	17.315	7.585	3.522	

(a)

HIDE	6	22	66	77
6		8.000	4.200	11.400
22	8.000		13.100	6.500
66	4.200	13.100		10.800
77	11.400	6.500	10.800	

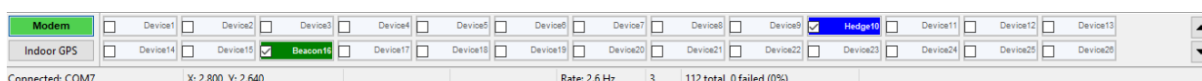
(b)

HIDE	5	22	66	77
5		7.144	12.389	10.101
22	7.144		10.122	12.151
66	12.389	10.122		6.879
77	10.101	12.151	6.879	

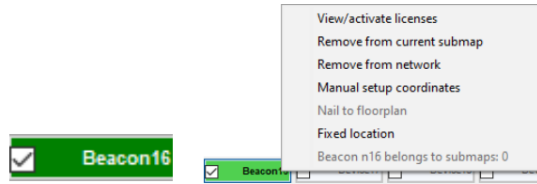
(c)

Hình 1.26: Bảng khoảng cách

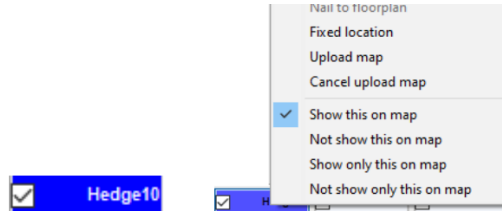
- Màu trắng - là mọi thứ đều tốt, bạn có thể đóng băng bản đồ
 - Màu vàng - là một cái gì đó có vẻ sai, kiểm tra khoảng cách các beacons trước khi đóng băng bản đồ.
 - Màu đỏ - là một sai số khoảng cách quan trọng, không đóng băng bản đồ. Đo thử công và nhập khoảng cách.
 - Màu xanh lá cây - là khoảng cách đã được "pair freeze", có thể đóng băng bản đồ.
- Danh sách các thiết bị: Danh sách thiết bị chứa thông tin về tất cả các beacons trong hệ thống được minh họa trong Hình 1.27. Nó cũng cho phép tìm kiếm, thêm và xóa bỏ.

**Hình 1.27:** Danh sách các thiết bị có trong hệ thống

Các thiết bị trong phần này được chia thành hai loại như Hình 1.28 gồm: Beacon cố định (beacon) và Beacon di động (hedge).



(a) Beacon



(b) Hedge

Hình 1.28: Hai loại beacon

- Thanh công cụ cài đặt: Thanh này cho phép người dùng điều chỉnh thiết bị rất chính xác. Nó chứa rất nhiều tham số cho việc sử dụng nâng cao.

CPU ID	Copy to clipboard	152437	ID bộ xử lý cho mỗi thiết bị (beacon hoặc modem)
Firmware version		v7.046 Modem HW v5.1	Firmware version của thiết bị
Architecture		NIA	Kiểu kiến trúc (IA hoặc NIA)
Location update rate		8 Hz	Cài đặt tốc độ cập nhật vị trí: 0.5Hz - 16Hz+. Tốc độ cập nhật thực có thể bị giới hạn bởi khoảng cách các beacons
Supply voltage, V (4.5..5.5)		4.87	Nguồn cung cấp của thiết bị 5V+-0.2V
Time from reset, h:m:s		00:59:13 / 15:41.48 / 0	Thời gian từ lần cuối thực hiện reset lại thiết bị
RSSI from 010, dBm		-50	
RSSI to 010, dBm		-49	
Profile		General	
Radio frequency band		"915 MHz"	
Carrier frequency, MHz		919.0	
Radio channel		0	Cường độ của tín hiệu từ modem đến beacons và ngược lại. Duy trì trong khoảng từ -25dBm đến -80dBm. Cao hơn có thể quá tải, thấp hơn có thể mất dữ liệu
Device address (1..254)		1	
Long time sleep		disabled	
Window of averaging (0..16)		4	
Distance filter (0..16)		0	
Max. hedges readout per cycle (1..255)		n/a	
High resolution mode (mm)		enabled	
Detect new beacons on frozen map		enabled	
Temperature of air, °C (-20..60)		23	
Update location mode		License SW v7.1 required	

(a)

CPU ID	Copy to clipboard	152437	
Firmware version		v7.046 Modem HW v5.1	
Architecture		NIA	
Location update rate		8 Hz	
Supply voltage, V (4.5..5.5)		4.88	
Time from reset, h:m:s		00:00:27 / 17:01.04 / 0	
RSSI from 009, dBm		-37	
RSSI to 009, dBm		-33	
Profile		General	
Radio frequency band		"915 MHz"	
Carrier frequency, MHz		919.0	
Radio channel		0	Kênh được chọn trước, một trong các kênh cho việc truyền thông giữa modem và beacons
Device address (1..254)		1	Địa chỉ logic của thiết bị từ 1-254
Long time sleep		disabled	TBD
Window of averaging (0..16)		4	Trung bình các phép đo cập nhật vị trí. Càng nhiều giá trị thì sai số càng ít nhưng độ trễ sẽ cao
Distance filter (0..16)		0	
Max. hedges readout per cycle (1..255)		n/a	
High resolution mode (mm)		enabled	Bộ lọc khoảng cách
Detect new beacons on frozen map		enabled	
Temperature of air, °C (-20..60)		23	
Update location mode		License SW v7.1 required	

(b)

Hình 1.29: Thanh công cụ cài đặt

1.3 Bài tập thực hành

1.3.1 Bài tập 1.1: Làm quen với môi trường mô phỏng VREP

- **Thiết lập thí nghiệm:**

- Giải nén *baitap1.rar* vào thư mục *local_folder*.
- Chạy phần mềm Copperliasim và nạp file scene *local_folder/bot*.
- Bật Matlab, chuyển đến *local_folder* và chạy file *simpletest.m*. Nếu không có bất kỳ thông báo lỗi nào và Copperliasim thực hiện mô phỏng thì việc thiết lập được xem là thành công. Trường hợp có lỗi, sinh viên cần lưu ý đảm bảo file *remApi.m* và *remoteApiProto.m* có trong thư mục hiện tại của matlab.

- **Yêu cầu:**

- Viết chương trình trên matlab điều khiển chuyển động của robot.
- Viết chương trình trên matlab đọc dữ liệu cảm biến của robot.

1.3.2 Bài tập 1.2: Mô phỏng Robot tránh vật cản trên VREP

- **Thiết lập thí nghiệm:**

- Giải nén *baitap2.rar* vào thư mục *local_folder*.
- Chạy phần mềm Copperliasim và nạp file scene *local_folder/bot*.
- Bật Matlab, chuyển đến *local_folder* và mở file *obsavoid.m*.

- **Yêu cầu:**

- Viết chương trình trên matlab điều khiển robot tránh vật cản chỉ sử dụng cảm biến phía trước.
- Viết chương trình trên matlab điều khiển robot tránh vật cản sử dụng cả 5 cảm biến.

1.3.3 Bài tập 1.3: Làm quen với phần cứng robot thực

- **Thiết lập thí nghiệm:**

- Kiểm tra kết nối phần cứng và nguồn pin nuôi robot đảm bảo khoảng 12V.

- Kết nối bộ điều khiển trung tâm của robot (arduino) với máy tính qua cổng USB.
- Chạy Arduino IDE, mở file *simpletest.ino*, biên dịch và nạp chương trình cho robot. Chương trình ví dụ có nhiệm vụ điều khiển robot đi thẳng và dừng sau 2 giây.
- Ngắt kết nối với máy tính, đặt robot lên sàn nhà, bật nguồn để robot hoạt động và quan sát kết quả.

- **Yêu cầu:**

- Viết chương trình trên arduino điều khiển chuyển động của robot
- Viết chương trình trên arduino đọc dữ liệu cảm biến của robot

1.3.4 Bài tập 1.4: Thí nghiệm tránh vật cản với robot thực

- **Thiết lập thí nghiệm:**

- Chạy Arduino IDE, mở file *obsavoid.ino* và viết chương trình điều khiển robot theo yêu cầu bên dưới.
- Kết nối robot với máy tính qua cổng USB. Biên dịch và nạp chương trình cho robot.
- Ngắt kết nối với máy tính, đặt robot lên sàn nhà, bật nguồn để robot hoạt động và quan sát kết quả.

- **Yêu cầu:**

- Viết chương trình trên arduino điều khiển robot tránh vật cản chỉ sử dụng cảm biến phía trước.
- Viết chương trình trên arduino điều khiển robot tránh vật cản sử dụng cả 5 cảm biến.

1.3.5 Bài tập 1.5: Hệ thống định vị trong nhà MarvelMind

- **Thiết lập thí nghiệm:**

- Thiết lập vị trí các beacon cố định trên một độ cao có thể nhìn thấy robot.
- Tích hợp beacon di động trên mô hình robot thực tế.

- Chạy trình duyệt dashboard, kết nối Modern với máy tính thông qua dây nối USB.

- **Yêu cầu:**

- Thiết lập vị trí chính xác các beacon hiển thị trong ứng dụng Dashboard.
- Lấy được vị trí của beacon nằm trên robot.
- Chạy chương trình trên Arduino IDE, mở file *hedgehog.ino* và viết chương trình lấy được thông tin vị trí của beacon nằm trên robot thông qua giao thức Serial. Sau đó đánh giá sai số phép đo thông qua quá trình đo trực tiếp bằng tay.

MÔ HÌNH ĐỘNG HỌC ROBOT 2 BÁNH VI SAI

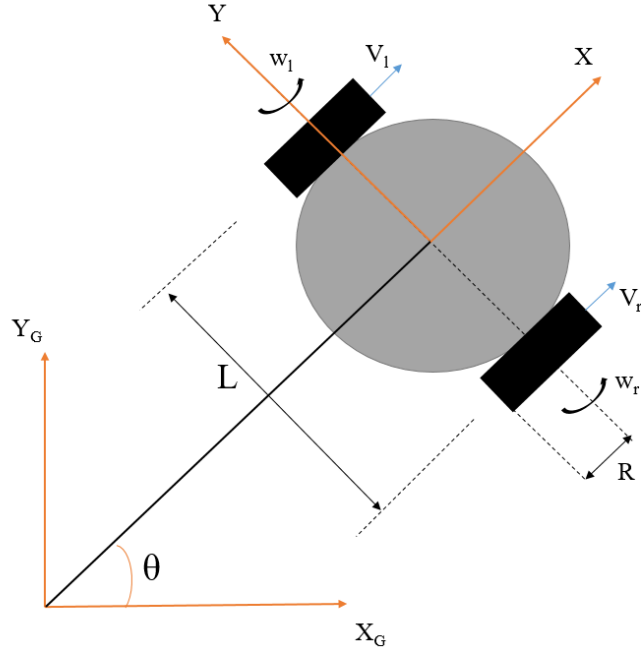
Mục đích: Bài này giúp sinh viên hiểu và áp dụng mô hình động học để điều khiển robot 2 bánh vi sai. Sau bài học, sinh viên có thể:

- Hiểu được phương trình động học thuận và động học ngược của mô hình robot hai bánh vi sai.
- Lập trình thử nghiệm các mô hình động học trên robot mô phỏng và robot thật.

2.1 Động học thuận

Mục đích của bài toán động học thuận trong robot di động là xác định vị trí và hướng của robot dựa trên góc quay của các bánh xe phát động. Thông số động học của robot 2 bánh vi sai sử dụng như mô tả trong Hình 2.1 như sau:

- L là độ dài trục hoặc khoảng cách giữa 2 bánh xe của robot.
- R là bán kính bánh xe.
- θ là góc hướng hiện tại của robot.
- (x, y) là tọa độ của Robot trong khung tham chiếu gốc.
- V là vận tốc dài của robot.
- ω là vận tốc góc/xoay của robot.
- V_l và V_r lần lượt là vận tốc dài của bánh trái và bánh phải của robot.
- ω_l và ω_r lần lượt là vận tốc góc của bánh trái và bánh phải của robot. Nếu xét trên một đơn vị thời gian thì hai tham số này tương ứng với góc quay của 2 bánh xe phát động - tham số đầu vào cho bài toán động học thuận của robot.



Hình 2.1: Mô hình của robot

Bài toán động học thuận cho robot hai bánh vi sai có thể phát biểu là: tìm vị trí mới của robot bao gồm (x, y, θ) nếu biết vận tốc góc quay của hai bánh xe ω_l và ω_r .

Từ mô hình của robot ta có được biểu diễn mô hình bằng phương trình sau:

$$\begin{cases} \dot{x} = V \cos \theta \\ \dot{y} = V \sin \theta \\ \dot{\theta} = \omega \end{cases} \quad (2.1)$$

Với $\dot{x}$, $\dot{y}$ tương ứng là vận tốc tuyến tính của robot theo trục x, y ; $\dot{\theta}$ là vận tốc góc/xoay của robot $\dot{\theta} = \omega$.

Vì:

$$V = \frac{R}{2}(\omega_r + \omega_l) \quad (2.2)$$

$$\omega = \frac{R}{2}(\omega_r - \omega_l) \quad (2.3)$$

Thay vào phương trình 2.1 ta có được phương trình mới:

$$\begin{cases} \dot{x} = \frac{R}{2}(\omega_r + \omega_l) \cos \theta \\ \dot{y} = \frac{R}{2}(\omega_r + \omega_l) \sin \theta \\ \dot{\theta} = \frac{R}{2}(\omega_r - \omega_l) \end{cases} \quad (2.4)$$

Phương trình 2.4 có thể được biểu diễn lại dưới dạng ma trận như sau:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{R}{2} \cos \theta & \frac{R}{2} \cos \theta \\ \frac{R}{2} \sin \theta & \frac{R}{2} \sin \theta \\ \frac{R}{2} & -\frac{R}{2} \end{bmatrix} \begin{bmatrix} \omega_l \\ \omega_r \end{bmatrix} \quad (2.5)$$

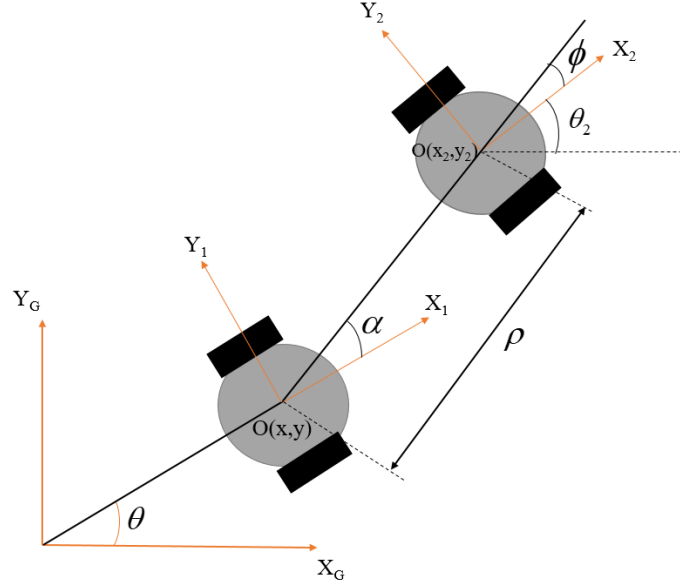
Để có được vị trí mới của robot tại thời điểm $t + 1$ từ vị trí cũ và vận tốc của robot tại thời điểm t trong một vòng lặp điều khiển $(\dot{x}, \dot{y}, \dot{\theta})$ với bước thời gian Δt ta có thể dùng một phép ước tính tuyến tính đơn giản như sau:

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} + \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} \Delta t \quad (2.6)$$

2.2 Động học ngược

Ngược lại với động học thuận, động học ngược là vấn đề đi tìm vận tốc góc ω_l, ω_r của hai bánh xe robot, biết vị trí mong muốn là (x, y, θ) . Tuy nhiên đối với mô hình robot 2 bánh vi sai hoặc robot dạng ô tô, việc giải động học ngược là không thể do các mô hình này vướng vào ràng buộc *non-holonomic*, hoặc có thể coi là ràng buộc không thể tích phân (sinh viên có thể tự tìm hiểu thêm để chứng minh ràng buộc này là đúng). Ví dụ, một chiếc ô tô có thể đạt đến bất kỳ vị trí cuối cùng nào trong mặt phẳng của nó, nhưng nó không bao giờ có thể di chuyển sang ngang. Do đó, tùy thuộc vào vị trí mục tiêu, nó yêu cầu thực hiện một loạt thao tác (chẳng hạn như đỗ xe song song) để đạt được trạng thái mong muốn. Để thay cho việc không thể giải được động học ngược, chúng ta có thể sử dụng các thuật toán lập kế hoạch chuyển động (motion planning). Tuy nhiên trong chương trình của thuật toán này chúng ta sẽ thực hiện phương pháp điều khiển vị trí tới vị trí (Pose to Pose) sử dụng phương pháp được đề xuất trong trang 92 của sách

giáo trình *Introduction to Autonomous Mobile Robots* của tác giả Roland Siegwart và các cộng sự. Phương pháp này được thực hiện đơn giản như sau:



Hình 2.2: Điều khiển Pose tới Pose

Giả sử một robot có vị trí ban đầu là (x, y, θ) và vị trí mục tiêu là (x_2, y_2, θ_2) . Chúng ta sẽ chuyển robot từ hệ tọa độ Đề các sang hệ tọa độ cực với gốc tọa độ là vị trí (x_2, y_2) , như vậy ta có được:

$$\begin{cases} \rho = \sqrt{(x_2 - x)^2 + (y_2 - y)^2} \\ \phi = \arctan 2(y_2 - y, x_2 - x) - \theta_2 \\ \alpha = \arctan 2(y_2 - y, x_2 - x) - \theta \end{cases} \quad (2.7)$$

Điều này có nghĩa là khi càng tiến gần đến vị trí mong muốn, (ρ, ϕ, α) sẽ tiến tới $(0, 0, 0)$. Và cuối cùng, để điều khiển vận tốc của robot chúng ta sẽ sử dụng một bộ điều khiển hệ số đơn giản như sau:

$$\begin{cases} V = K_\rho \rho \\ \omega = K_\alpha \alpha + K_\phi \phi \end{cases} \quad (2.8)$$

Với K_ρ , K_α , K_ϕ là các hệ số điều chỉnh, sinh viên sẽ phải tự tìm ra bộ hệ số điều khiển phù hợp.

Từ phương trình 2.4, chúng ta có thể tính được vận tốc góc hai bánh xe của robot ω_r, ω_l theo giá trị V và ω được xác định trong phương trình 2.8:

$$\begin{cases} \omega_r = \frac{V + \omega \frac{L}{2}}{R} \\ \omega_l = \frac{V - \omega \frac{L}{2}}{R} \end{cases} \quad (2.9)$$

2.3 Bài tập thực hành

2.3.1 Bài tập 2.1 Mô phỏng động học thuận robot

Sinh viên mở file ***forwardKinematics.m*** trên Matlab, tìm đoạn chương trình còn trống trong file và hoàn thiện code cho bài toán động học thuận theo chỉ dẫn đã được chú thích trong file.

Trong file ***forwardKinematics.m*** sinh viên sẽ được cho trước vận tốc 2 bánh của robot trong 2 biến *omega_r* và *omega_l*, nhiệm vụ của sinh viên là sử dụng vận tốc 2 bánh đã cho để tính toán vị trí của robot sử dụng phương trình động học thuận, đường đi thực tế và đường đi ước tính sử dụng phương trình động học thuận phải tương đương nhau.

Sau khi hoàn thành, sinh viên chạy chương trình mô phỏng theo quy trình đã thực hiện ở chương 1 và quan sát kết quả.

2.3.2 Bài tập 2.2 Mô phỏng động học ngược robot

Sinh viên mở file ***pose2pose.m*** trên Matlab, tìm đoạn chương trình còn trống trong file và hoàn thiện code cho bài toán động học thuận theo chỉ dẫn đã được chú thích trong file.

Trong file ***pose2pose.m*** mục tiêu của sinh viên là sử dụng phương pháp đã được học để điều khiển robot tới một Pose mục tiêu (sinh viên có thể tự quy định). Các tham số điều khiển như *K_rho*, *K_alpha*, *K_phi* sinh viên sẽ phải tự tìm ra bộ tham số điều khiển phù hợp.

Sau khi hoàn thành, sinh viên chạy chương trình mô phỏng theo quy trình đã thực hiện ở chương 1 và quan sát kết quả.

2.3.3 Bài tập 2.3 Thực nghiệm động học thuận

Sinh viên hoàn thiện code còn trống trong file ***forwardKinematics.ino*** tương tự như phần mô phỏng, sử dụng hàm **motorGo** trong thư viện **motorControl.h** để điều khiển động cơ.

Sau khi hoàn thành, sinh viên chạy nạp chương trình vào arduino và chạy thí nghiệm theo quy trình đã thực hiện ở chương 1 và quan sát kết quả. Sinh viên đánh giá sự khác và giống giữa thực nghiệm và mô phỏng, nếu có sự khác biệt thì đó là gì?

2.3.4 Bài tập 2.4 Thực nghiệm động học ngược

Sinh viên hoàn thiện code còn trống trong file ***poseToPose.ino*** tương tự như phần mô phỏng, sử dụng hàm **motorGo** trong thư viện **motorControl.h** để điều khiển động cơ.

Sau khi hoàn thành, sinh viên chạy nạp chương trình vào arduino và chạy thí nghiệm theo quy trình đã thực hiện ở chương 1 và quan sát kết quả. Sinh viên đánh giá sự khác và giống giữa thực nghiệm và mô phỏng, nếu có sự khác biệt thì đó là gì?

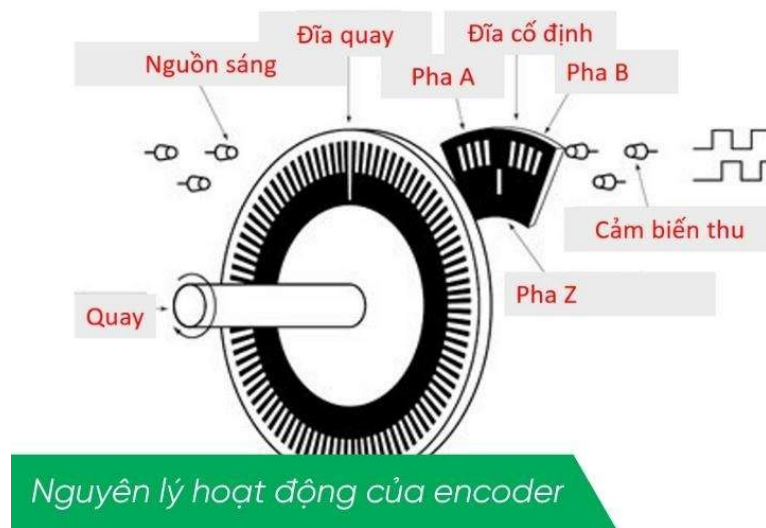
KỸ THUẬT ODOMETRY

Mục đích: Trong bài toán thực tế, việc quan trọng là Robot phải biết vị trí của mình ở đâu. Đây luôn là một bài toán thách thức và cần lời giải. Có nhiều cách để Robot có thể nhận thức được vị trí của bản thân như sử dụng các hệ thống định vị bên ngoài như GPS, Marvelmind hay sử dụng các cảm biến có trên robot (onboard sensor),... Trong đó Odometry là một phương pháp giúp robot xác định vị trí tương đối được bản thân trong môi trường xung quanh. Bài thực hành giúp sinh viên làm quen với cách tính toán vị trí tương đối của robot sử dụng phương pháp Odometry. Kết thúc bài học, sinh viên có thể:

- Hiểu được việc áp dụng các công thức Odometry trong mô phỏng. Dựa vào các lý thuyết đã được đề cập, sử dụng được công thức gọi được vận tốc của bánh xe. Từ đó tính toán ra vị trí của robot.
- Hiểu được cách lập trình quan sát chuyển động của xe nhờ quá trình bám xung. Học được cách sử dụng encoder trong robot thực tế, dựa vào đó tính toán được vị trí mà robot đạt được.

3.1 Phương pháp Odometry

Việc đo lường vị trí tương đối của robot so với vị trí ban đầu bằng cách sử dụng các cảm biến chuyển động như encoder được gọi là phương pháp Odometry. Ví dụ, nếu một robot di chuyển thẳng về phía trước và biết được đường kính bánh xe, robot có thể tự xác định được nó đã di chuyển bao xa so với vị trí ban đầu bằng cách đếm số lượng vòng xoay của bánh xe. Bánh xe được trang bị bộ lập mã trực (encoder) với số vạch định trước tương ứng với số xung được tạo ra trong mỗi vòng quay của mô tơ như minh họa trong Hình 3.1.



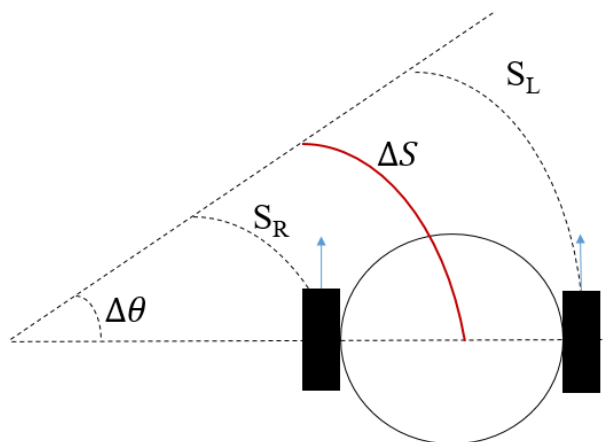
Hình 3.1: Mô hình hoạt động của Encoder

Điều kiện của thuật toán Odometry trên thực tế:

- Biết bán kính bánh xe.
- Bộ cảm biến ước lượng vận tốc Robot, chẳng hạn đĩa lập mã (encoder) hoặc cảm biến gia tốc (accelerometer), ...

Thông thường trên thực tế từ encoder ta có thể dễ dàng tính được vận tốc quay của bánh xe tuy vậy khác với thực tế, trên Vrep ta có thể dễ dàng mô phỏng điều này sử dụng câu lệnh trực tiếp lấy được vận tốc quay w_L và w_R của bánh xe.

Thiết lập Odometry:



Hình 3.2: Mô hình mô tả Odometry

Bước đầu ta tính quãng đường thay đổi S_L và S_R của Robot

$$\begin{cases} S_R = w_R Rdt \\ S_L = w_L Rdt \\ \Delta S = \frac{S_R + S_L}{L} \end{cases} \quad (3.1)$$

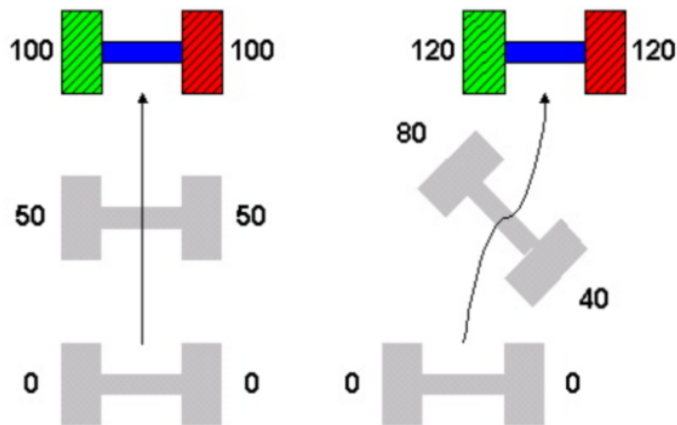
Bước tiếp theo tính góc mà Robot thay đổi:

$$\Delta\theta = \frac{S_R - S_L}{L} \quad (3.2)$$

Bước cuối cùng là cập nhật lại vị trí của Robot dựa trên mô hình động học:

$$\begin{cases} x_{t+1} = x_t + \Delta S \cos(\theta) \\ y_{t+1} = y_t + \Delta S \sin(\theta) \\ \theta_{t+1} = \theta_t + \Delta\theta \end{cases} \quad (3.3)$$

Tuy tiện lợi và dễ sử dụng nhưng Odometry có những hạn chế, Odometry sử dụng các cảm biến đo lường chuyển động của bánh xe. Bất kỳ sự thiếu chính xác nào trong cảm biến sẽ trở nên tồi tệ theo thời gian vì đó là một phép đo tích lũy. Lỗi suất hiện làm phương pháp Odometry trở nên sai lệch có thể bị gây ra bởi đo lường sự sai của các hệ thống truyền động nhiều bánh có đường kính bánh xe với các vấn đề về đường kính bánh xe khác nhau trong việc đếm xung trong khi sử dụng Encoder trong robot. Để tránh tích tụ các lỗi, robot đôi khi sẽ cần sử dụng các cảm biến bổ sung để định vị trí chính xác của chúng.



Hình 3.3: Mô tả cách tính Phương pháp Odometry

Một động cơ hai bánh vì sai di chuyển theo hai đường theo hình 3.3 (mỗi bánh xe được kích hoạt bởi một động cơ riêng). Các giá trị của bộ mã hóa encoder

được đặt bên cạnh các bánh xe. Hình bên trái, robot di chuyển thẳng, do đo được giá trị các encoder mỗi bánh luôn giống nhau. Hình bên phải, cho thấy rằng robot đã rẽ phải trong một phần của quãng đường di chuyển với bộ mã hóa encoder đọc được bánh trái có giá trị 80 và bánh phải có giá trị 40. Tuy nhiên, các bộ mã hóa ở vị trí cuối cùng của robot đã hiển thị cùng một giá trị vì vậy hình dạng di chuyển của xe mang hình chữ S. Nếu sinh viên chỉ chú ý đến giá trị đọc của các encoder tại giá trị cuối cùng, sẽ tưởng gần như hai xe ở hai hình đi với hình dạng giống nhau đều đi thẳng. Điều này dẫn đến sai lầm trong quá trình xử lý.

3.2 Hướng dẫn điều khiển Robot sử dụng encoder trong Coppeliassim

Câu lệnh thực hiện gọi trực tiếp vận tốc của mỗi bánh xe *simxGetObjectVelocity* được mô tả như hình bên dưới:

simxGetObjectVelocity	
Description	Retrieves the linear and angular velocity of an object. See also simxGetObjectPosition , simxGetObjectOrientation and simxGetObjectGroupData .
Matlab synopsis	[number returnCode,array linearVelocity,array angularVelocity]=simxGetObjectVelocity(number clientID,number objectHandle,number operationMode)
Matlab parameters	clientID : the client ID, refer to simxStart . objectHandle : handle of the object operationMode : a remote API function operation mode . Recommended operation modes for this function are simx_opmode_streaming (the first call) and simx_opmode_buffer (the following calls)
Matlab return values	returnCode : a remote API function return code linearVelocity : 3 values representing the linear velocity (vx, vy, vz). angularVelocity : 3 values representing the angular velocity (dAlpha, dBeta, dGamma).
Other languages	C/C++ , Python , Java , Octave

Hình 3.4: Lấy vận tốc bánh xe trong phần mềm mô phỏng Coppeliassim

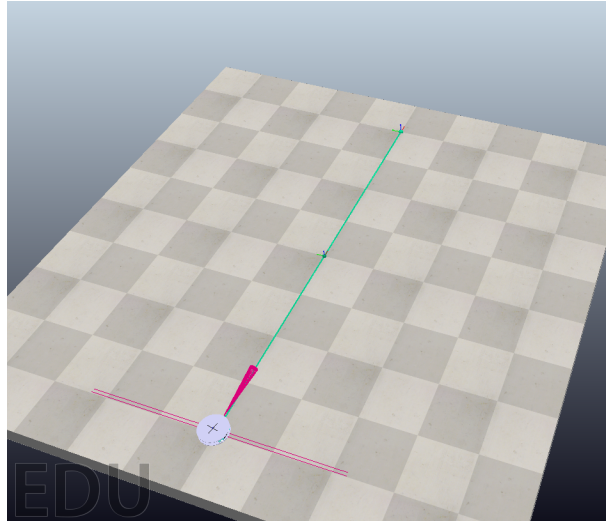
3.3 Bài tập thực hành

3.3.1 Bài tập mô phỏng

Bài tập 3.1: Thí nghiệm đọc giá trị Encoder

Dựa vào câu lệnh đã được cung cấp trong bài đã học đưa robot đi thẳng tới vị trí đích như Hình 3.5. Sinh viên viết đoạn chương trình để ước lượng vị trí tương đối của robot sử dụng phương pháp Odometry.

Sau khi hoàn thành, sinh viên chạy chương trình mô phỏng theo quy trình đã thực hiện ở chương 1 và quan sát kết quả.



Hình 3.5: Bài tập 1: Robot di chuyển theo đường thẳng

Bài tập 3.2: Sử dụng Odometry định vị tương đối

Điều khiển robot di chuyển theo hình Vuông với độ dài cạnh 1m. Sử dụng phương pháp Odometry để ước lượng vị trí tương đối của robot.

Sau khi hoàn thành, sinh viên chạy chương trình mô phỏng theo quy trình đã thực hiện ở chương 1 và quan sát kết quả.

3.3.2 Bài tập thực nghiệm

Bài tập 3.3: Thí nghiệm đếm giá trị của Encoder

- **Thiết lập thí nghiệm:**

- Tải xuống vào giải nén file **Odometry.rar**, truy cập folder readEncoder, mở file *readEncoder.ino*.
- Sinh viên sử dụng robot thí nghiệm, kết nối robot với máy tính để tải chương trình lên board Arduino.

- **Yêu cầu:**

- Trong tệp tin **ReadEncoder.ino**, lệnh interrupt cho các chân đếm encoder, sử dụng hàm *"attachInterrupt"* để xử lý ngắt tương ứng cho hai bánh trái và phải với từng chân encoder và xác định xung ngắt để tăng giá trị đếm encoder. Để tăng giá trị đếm encoder, cần hoàn thiện code trong hàm *"attachInterrupt"*.

- Đọc giá trị vị trí của encoder mỗi bánh khi mỗi bánh quay theo *chiều dương* với giá trị xung **PWM = 50** trong **2** giây.

Bài tập 3.4: Sử dụng phương pháp Odometry

Sinh viên sẽ hoàn thiện phần code còn trống trong file ***odometryRobot.ino*** với mục tiêu hiểu cách sử dụng phương pháp Odometry, sử dụng hàm *motorGo* trong lớp Odometry để điều khiển động cơ trong thư viện ***"odometry.h"***. Đánh giá kết quả chạy Odometry trong thực tế và các vấn đề khó khăn khi làm thực nghiệm.

- **Thiết lập thí nghiệm:**

- Truy cập folder Odometry, mở file *Odometry.ino*.
- Sinh viên sử dụng robot thí nghiệm, kết nối robot với máy tính để tải chương trình lên board Arduino.

- **Yêu cầu:**

- Trong file ***odometry.ino***, sinh viên đo biến đếm của encoder nhận được khi động cơ quay được một vòng điền vào hàm *getCpc()*.
- Sinh viên vẽ lưu đồ thuật toán Odometry.

THUẬT TOÁN DẪN ĐƯỜNG BUG

Mục đích: Bài này giúp sinh viên tìm hiểu và áp dụng thuật toán dẫn đường cho robot di động hai bánh vi sai. Kết thúc bài học, sinh viên có thể:

- Hiểu được thuật toán dẫn đường Bug-1 và Bug-2.
- Lập trình và thử nghiệm thuật toán Bug-1 và Bug-2 trên robot mô phỏng và robot thật.

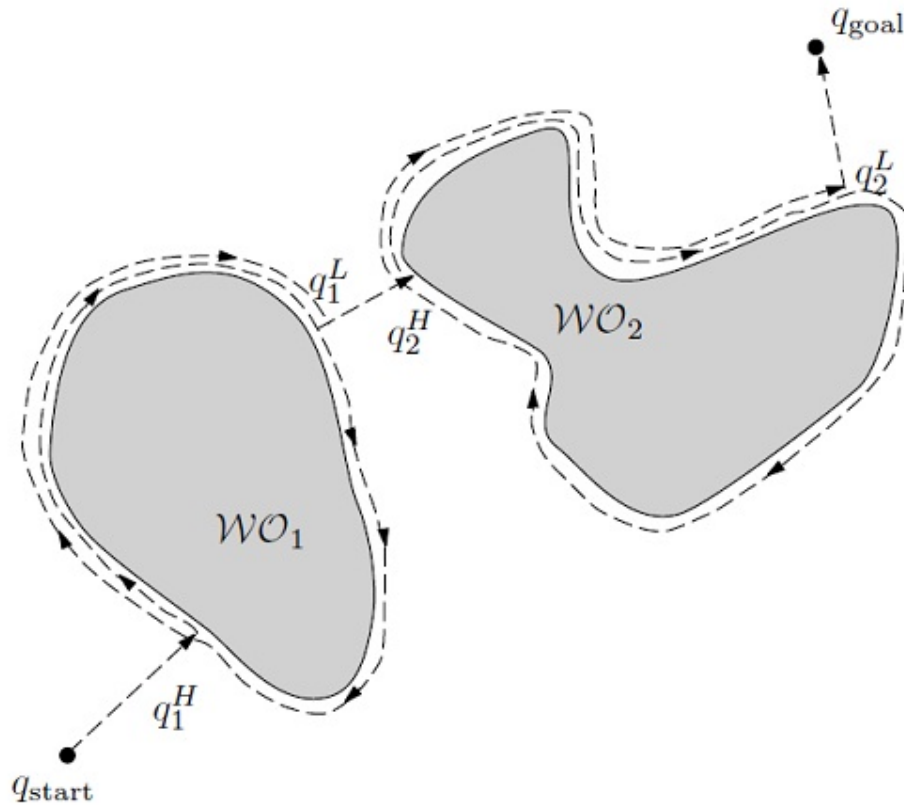
Thuật toán Bug là một trong những thuật toán lập đường đi dựa trên cảm biến cục bộ đơn giản nhất. Các thuật toán này giả định rằng robot hoạt động trong một mặt phẳng và được trang bị các cảm biến tiệm cận. Chúng yêu cầu robot có hai hành vi: di chuyển tiến tới đích và bám tường. Trong chương này, thuật toán Bug-1 và Bug-2 cùng với hai hành vi nói trên sẽ được trình bày.

4.1 Bug-1

Bug-1 là thuật toán tránh chướng ngại vật ra đời sớm nhất, dễ điều chỉnh và không bị ảnh hưởng bởi cực tiểu cục bộ tuy nhiên trong một số trường hợp nó lại đưa robot ra xa mục tiêu. Trong thuật toán này, robot sau khi phát hiện chướng ngại vật sẽ bắt đầu đi theo đường biên của chướng ngại vật cho đến khi quay lại điểm mà robot bắt đầu đi theo bám tường, Trong quá trình đó robot luôn tính toán và lưu lại điểm có khoảng cách gần với đích đến nhất, điểm này được gọi là điểm rời. Sau khi robot đã đi một vòng quanh đường biên của chướng ngại vật và quay lại điểm bắt đầu bám tường, robot sẽ tiếp tục bám tường cho tới khi nào tới điểm rời. Cuối cùng, khi tới được điểm rời robot sẽ thực hiện hành vi Tiến tới đích. Quá trình trên lặp lại tới khi robot tới được mục tiêu hoặc robot nhận ra mọi đường đi tới đích đều bị chặn.

Quá trình trên có thể ngắn gọn lại như sau:

1. Di chuyển hướng tới đích.
2. Nếu gặp chướng ngại vật, đi vòng quanh chướng ngại vật và liên tục lưu lại điểm có khoảng cách gần với mục tiêu nhất (điểm rời).
3. Quay lại điểm rời.
4. Lặp lại bước đầu.



Hình 4.1: Đường đi của thuật toán Bug-1

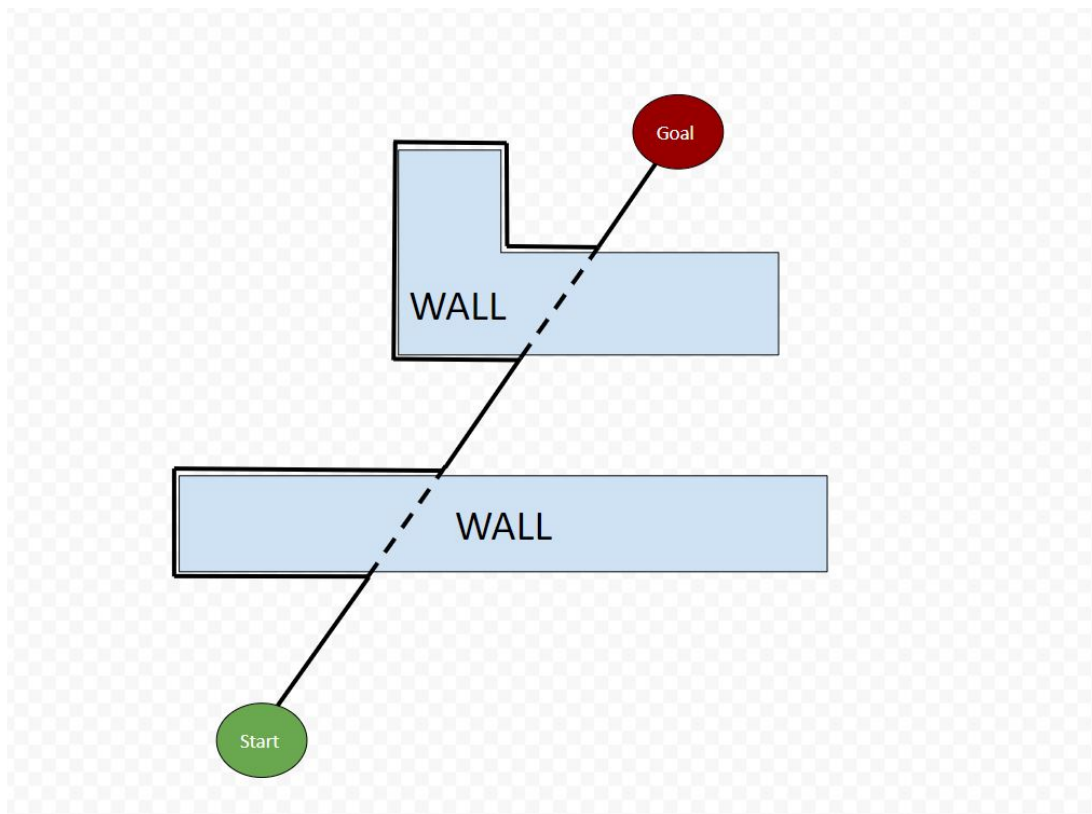
Một trong những nhược điểm phổ biến của thuật toán Bug-1 là khi robot đi theo rìa của chướng ngại vật 1, nó có thể va chạm với chướng ngại vật lân cận 2 trong trường hợp chướng ngại vật thứ 2 ở rất gần chướng ngại vật đầu tiên hoặc khoảng cách giữa chúng nhỏ hơn chiều rộng của robot.

4.2 Bug-2

Thuật toán Bug-2 là phiên bản cải tiến của Bug-1, thuật toán này tạo ra một đường dẫn dưới dạng một đường thẳng d được tạo bởi điểm bắt đầu P_s và điểm đích P_g . Robot được bắt đầu với hành vi đi tới đích và chuyển sang hành vi bám tường khi gặp vật cản. Robot sẽ giữ trạng thái bám tường cho tới khi vị trí của robot này nằm trên đường thẳng d một lần nữa, robot sẽ chuyển sang hành vi tiến tới đích. Quá trình này lặp lại cho tới khi robot tới được mục tiêu.

Các bước thực hiện Bug-2 :

1. Di chuyển hướng tới đích.
2. Nếu gặp chướng ngại vật, đi vòng quanh chướng ngại vật tới khi chạm đường thẳng d .
3. Lặp lại bước 1.



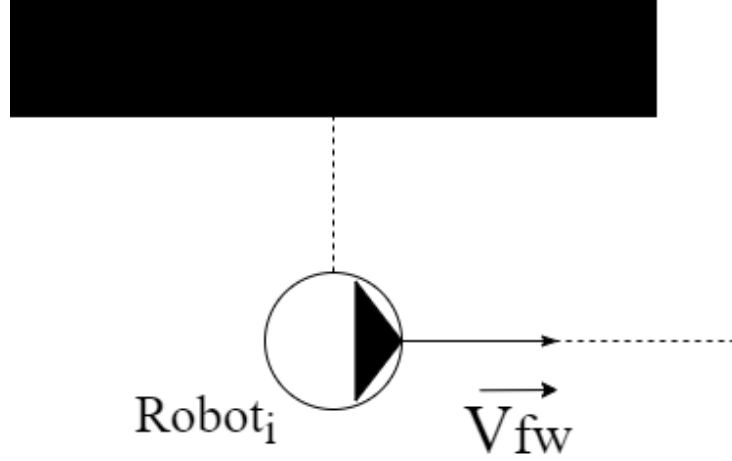
Hình 4.2: Đường đi của thuật toán Bug-2

Cả thuật toán Bug-1 và Bug-2 đều yêu cầu bộ nhớ tối thiểu. Tuy nhiên, chúng không có khả năng sử dụng tối ưu dữ liệu cảm biến để tạo ra các đường dẫn ngắn

hơn. Một cách tiếp cận cải tiến có tên là thuật toán Dist-Bug giải quyết vấn đề này.

4.3 Điều khiển hành vi

4.3.1 Hành vi bám tường



Hình 4.3: Hành vi bám tường

Khi môi trường trở nên phức tạp hơn, robot không thể hoàn thành nhiệm vụ nếu chỉ dựa vào hai hành vi *đi tới đích* và *phân tách* như ta thường thấy trong thuật toán Flocking của Craig Reynolds. Vì vậy, để tránh các vật cản lớn, hành vi bám tường là cần thiết. Hành vi này kích hoạt khi khoảng cách giữa robot và vật cản đạt tới một giá trị nhất định. Nếu điểm mục tiêu nằm ở bên trái của robot, chọn đi theo bên trái tường. Nếu điểm mục tiêu nằm ở bên phải của robot, chọn đi theo bên phải tường. Giả sử tọa độ hiện tại của robot là (x_i, y_i) , tọa độ của vật cản gần nhất phát hiện được là (x_0, y_0) . Vector đầu ra của hành vi bám tường là:

$$V_{fw} = \frac{1}{\sqrt{(x_0 - x_i)^2 + (y_0 - y_i)^2}} \begin{bmatrix} \pm(y_0 - y_i) \\ \mp(x_0 - x_i) \end{bmatrix} \quad (4.1)$$

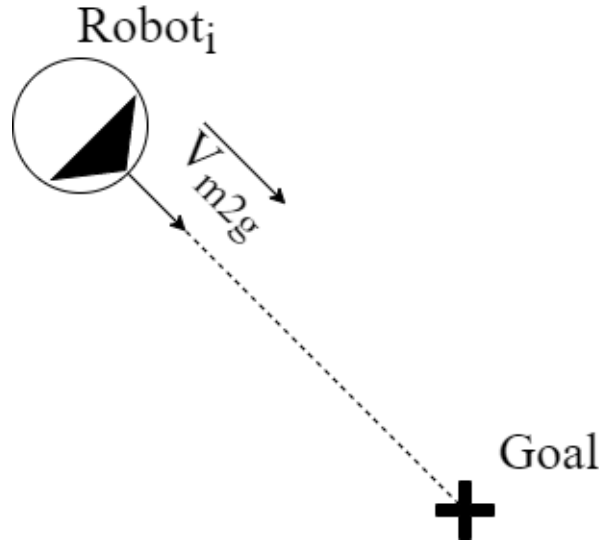
Dấu $\pm$ được xác định bởi mối quan hệ giữa hướng quay đầu của robot và chướng ngại vật. Nếu hướng của vật cản nằm ở bên trái ta chọn $+$. Ngược lại, nếu vật cản nằm bên phải ta chọn $-$. Tuy mang tên là bám tường, nhưng hành vi này có thể được sử dụng hiệu quả đối với các vật cản tĩnh với các kích cỡ từ nhỏ đến lớn, thay thế cho hành vi tránh vật cản thông thường. Cách tránh vật cản truyền

thông là tạo ra một Vector có hướng ngược lại với hướng của vật cản, nhằm đẩy robot lùi ra xa vật cản, nhưng cách làm này đôi khi gây ra tình trạng kẹt khi di chuyển hoặc tạo ra di chuyển giật cục. Để giải quyết vấn đề này, vector được tạo ra sẽ có hướng vuông góc với hướng của vật cản. Độ lớn của lực ảo được tạo ra tác dụng lên robot sẽ tăng dần khi robot tiến gần tới vật cản. Do trước đó vector bám tường đã được biến đổi thành vector đơn vị có độ dài bằng 1, nên độ lớn của lực ảo sẽ được điều chỉnh thông qua hệ số điều khiển sau đây:

$$f_{fw}() = \begin{cases} 0 & \text{Nếu không có vật cản} \\ a_{fw} & \text{Nếu có vật cản} \end{cases} \quad (4.2)$$

Trong đó a_{fw} là một tham số có thể điều chỉnh.

4.3.2 Hành vi đi tới đích



Hình 4.4: Hành vi đi tới đích

Có thể thấy trong hình 4.4, nếu như không có vật cản robot sẽ thực hiện hành vi tiến tới mục tiêu với tốc độ tối đa. Trong hầu hết các trường hợp, robot duy trì hành vi di chuyển đến mục tiêu. Giả sử tọa độ hiện tại của robot là (x_i, y_i) và tọa độ của mục tiêu là (x_t, y_t) , vector hành vi của việc di chuyển đến mục tiêu là:

$$V_{m2g} = \frac{1}{\sqrt{(x_t - x_i)^2 + (y_t - y_i)^2}} \begin{bmatrix} (x_t - x_i) \\ (y_t - y_i) \end{bmatrix} \quad (4.3)$$

Hệ số điều khiển của hành vi đi tới đích được định nghĩa như sau:

$$f_{m2g}() = \begin{cases} 0 & \text{Nếu không có vật cản} \\ a_{m2g} & \text{Nếu có vật cản} \end{cases} \quad (4.4)$$

Trong đó a_{m2g} là tham số có thể điều chỉnh được.

4.4 Bài tập thực hành

4.4.1 Mô phỏng

Bài tập 4.1: Thuật toán Bug 1

Sinh viên hoàn thiện đoạn code còn thiếu trong file ***Bug_1.m*** và các file ***Wall_following.m*** và ***move2goal.m*** cho các hành vi bám tường, hướng mục tiêu.

Sau khi hoàn thành, sinh viên chạy chương trình mô phỏng theo quy trình đã thực hiện ở chương 1 và quan sát kết quả.

Bài tập 4.2: Thuật toán Bug 2

Sinh viên sẽ hoàn thiện đoạn code còn thiếu trong các file ***Bug_2.m***. Sau khi hoàn thành, sinh viên chạy chương trình mô phỏng theo quy trình đã thực hiện ở chương 1 và quan sát kết quả.

Lưu ý các hàm khác được sử dụng trong chương trình:

- *checkEncouteredTheLine.m*: kiểm tra liệu một điểm có nằm trên đường thẳng cho trước.
- *checkReachedWaypoint.m*: Kiểm tra robot đã tới một waypoint hay chưa.
- *constraintAngle.m*: đưa giá trị một góc về trong khoảng $-\pi$ và π .
- *distPoint2Line.m*: tính toán khoảng cách giữa 1 điểm và 1 đường thẳng.
- *getHeading.m*: lấy hướng quay hiện tại của robot.
- *isCornered.m*: kiểm tra liệu robot có bị vào góc hay không.
- *readSensors.m*: lấy dữ liệu của các cảm biến tiệm cận.
- *transformation.m*: chuyển đổi một Vector từ hệ tọa độ này qua hệ tọa độ khác.

4.4.2 Thực nghiệm

Bài tập 4.3: Thực nghiệm với thuật toán Bug 1 Tương tự như ở phần mô phỏng, sinh viên hoàn thiện các đoạn code còn trống sử dụng ngôn ngữ C++ trong file *Bug_1.ino* và các hàm *Wall_following* và *move2goal* chứa trong thư viện *functionsForBugAlgorithm.h*. Các hàm bổ sung tương tự như trong mô phỏng, tuy nhiên được viết bằng ngôn ngữ C++. Sinh viên được cung cấp các thư viện sau:

- **motorControl.h:** bao gồm các hàm để điều khiển động cơ.
- **readSensors.h:** bao gồm các hàm đọc cảm biến.
- **functionsForBugAlgorithm.h:** bao gồm các hàm để thực hiện thuật toán Bug.

Sau khi hoàn thành, sinh viên nạp chương trình cho arduino và thực hiện thí nghiệm theo quy trình trong chương 1. Sinh viên đánh giá sự khác và giống khi thực hiện thuật toán Bug 1 trong thực nghiệm và mô phỏng, nếu có sự khác biệt thì đó là gì?

Bài tập 4.4: Thực nghiệm với thuật toán Bug 2.

Tương tự bài tập 4.3, sinh viên hoàn thiện các đoạn code còn trống sử dụng ngôn ngữ C++ trong *Bug_2.ino*. Sau khi hoàn thành, sinh viên nạp chương trình cho arduino và thực hiện thí nghiệm theo quy trình trong chương 1. Sinh viên đánh giá sự khác và giống khi thực hiện thuật toán Bug 2 trong thực nghiệm và mô phỏng, nếu có sự khác biệt thì đó là gì?

KỸ THUẬT LẬP ĐƯỜNG ĐI

Mục đích: Bài này giúp sinh viên hiểu và thử nghiệm được các thuật toán lập đường đi cho robot. Kết thúc bài học, sinh viên có thể:

- Hiểu được cách thức vận hành của thuật toán tìm đường A^* và RRT.
- Hiểu được các lập trình, thử nghiệm thuật toán A^* và RRT cho robot trên môi trường mô phỏng và thực nghiệm.

5.1 Lập đường đi cho robot

Lập kế hoạch đường đi (Path Planning) cho robot là việc xác định tập hợp các điểm rời rạc trong không gian làm việc của robot kết nối điểm xuất phát với điểm đích, giúp robot có thể di chuyển theo các điểm đó để tiếp cận mục tiêu. Một số tham số được quan tâm trong bài toán này như sau: đường đi có độ dài nhỏ nhất, thời gian di chuyển ngắn nhất, khả năng tránh va chạm,...

Các thuật toán lập kế hoạch đường đi ta có thể kể đến như thuật toán A^* , Bezier, Dijkstra, thuật toán cây ngẫu nhiên RRT (Rapidly Random Tree), LQR Planner,... Trong chương này, chúng ta sẽ tìm hiểu và thực hành với 2 thuật toán điển hình và dễ tiếp cận nhất cho bài toán lập kế hoạch đường đi là giải thuật A^* , và RRT.

5.2 Thuật toán A^*

Giải thuật A^* là thuật toán tìm kiếm trong đồ thị. Thuật toán này tìm một đường đi từ một nút khởi đầu A tới một nút đích B cho trước (hoặc tới một nút thỏa mãn một điều kiện đích). Trên bản đồ có nhiều chướng ngại vật, việc tìm

đường từ điểm A đến B có thể khó khăn. Tuy nhiên, thuật toán A^* dùng phương pháp thử sai dựa trên kinh nghiệm (heuristic) tìm kiếm trên đồ thị thông thường. Về cơ bản là lập kế hoạch trước ở mỗi bước để đưa ra quyết định tối ưu hơn. Giải thuật A^* hoạt động bằng cách tạo một cây đường dẫn có chi phí thấp nhất từ nút bắt đầu đến nút đích. Điều làm cho A^* khác biệt và tốt hơn cho nhiều tìm kiếm là đối với mỗi nút, A^* sử dụng một hàm mục tiêu $f(n)$ đưa ra ước tính về tổng chi phí của một đường dẫn sử dụng từ điểm khởi đầu đến điểm đích. Hàm $f(n)$ được tính như sau:

$$f(n) = g(n) + h(n) \quad (5.1)$$

Trong đó:

$g(n)$: Chi phí di chuyển để di chuyển từ điểm bắt đầu đến một hình vuông nhất định trên lưới, theo đường được tạo ra để đến đó.

$h(n)$: Chi phí di chuyển ước tính để di chuyển từ hình vuông đã cho đó trên lưới đến đích cuối cùng. Giá trị $h(n)$ có 3 cách tính:

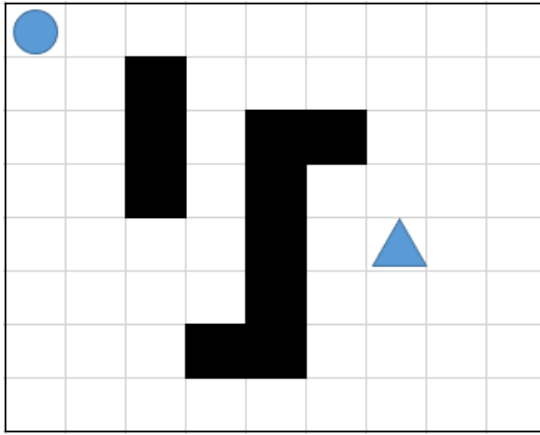
- Khoảng cách Matthan: Khoảng cách tính bằng tổng số hình vuông di chuyển thành chiều dọc và chiều ngang để đạt được vị trí mục tiêu.
- Khoảng cách Diagonal: Khoảng cách di chuyển theo đường chéo của các ô vuông để đến được vị trí mục tiêu.
- Khoảng cách Euclidean: Khoảng cách trực tiếp giữa vị trí hiện tại và vị trí mục tiêu.

Các bước tiến hành chạy thuật toán A^* :

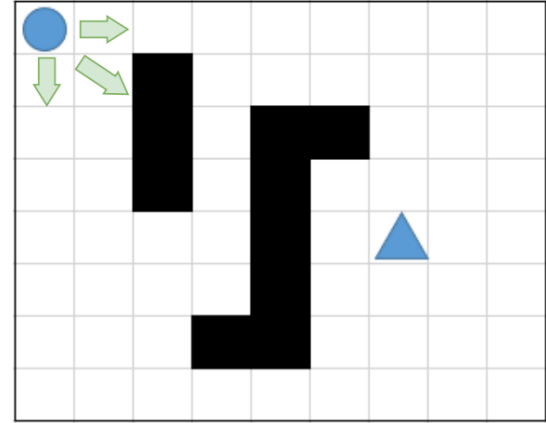
Bước 1: Khởi tạo bản đồ như minh họa trong Hình 5.1.a, Startnode và Endnode tương ứng là vị trí của các vật cản, vị trí ban đầu và vị trí đích.

Bước 2: Mở rộng các nút như minh họa trong Hình 5.1.b. Đặt Startnode là node parent và mở rộng ra các nút xung quanh từ node parent. **Bước 3:** Tính toán chi phí từ các node đó đến Endnode và chọn ra node có chi phí nhỏ nhất và không chứa vật cản. Thuật toán lần lượt tính các hàm $g(x)$ như Hình 5.1.c, $h(x)$ như Hình 5.1.d và $f(x)$ như Hình 5.1.e. Ở đây cách tính chi phí sẽ được tính theo cách tính Euclidean.

Bước 4: Cập nhật nút đó thành parent và tiếp tục vòng lặp cho đến khi đến được vị trí đích, từ đó xác định được đường đi ngắn nhất như minh họa trong Hình 5.1.f.



(a) Khởi tạo bản đồ



(b) Mở rộng nút

	1	2	3	4	5	6	7	8	
1	2			4	5	6	7	8	9
2	3			5			8	9	10
3	4			6			8	9	10
4	5	6	7			9		11	12
5	6	7	8			10	11	12	13
6	7	8				11	12	13	14
7	8	9	10	11	12	13	14	15	

(c) Tính chi phí $g(x)$

	2.2	3.2	4.1	5.1	6.1	7.1	8.1	9.1	
2.2	2.8		4.5	5.4	6.3	7.3	8.2	9.2	
3.2	3.6		5			7.6	8.5	9.5	
4.1	4.5		5.7			7.2	8.1	8.9	9.8
5.1	5.4	5.8	6.4			7.8	8.6	9.4	10
6.1	6.3	6.7	7.2			8.5	9.2	10	11
7.1	7.3	7.6				9.2	9.9	11	11
8.1	8.2	8.5	8.9	9.4	10	11	11	12	

(d) Tính chi phí $h(x)$

	0	1	2	3	4	5	6	7	8
0	3.2	5.2	7.1	9.1	11	13	15	17	
1	3.2	4.8		8.5	10	12	14	16	18
2	5.2	6.6		10		16	18	19	
3	7.1	8.5		12		15	17	19	21
4	9.1	10	12	13		17		20	22
5	11	12	14	15		18	20	22	24
6	13	14	16			20	22	24	25
7	15	16	18	19	20	22	24	25	27

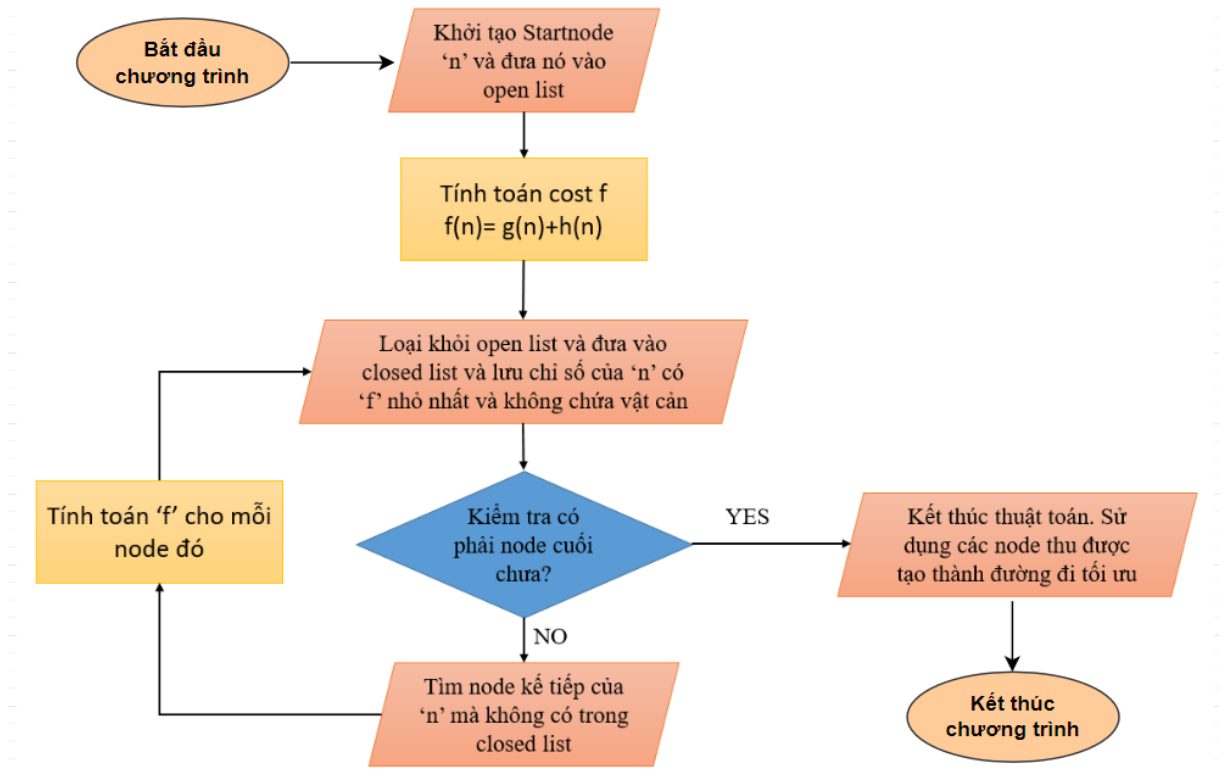
(e) Tổng chi phí $f(x)$

	0	1	2	3	4	5	6	7	8
0	3.2	5.2	7.1	9.1	11	13	15	17	
1	3.2	4.8		8.5	10	12	14	16	18
2	5.2	6.6		10		16	18	19	
3	7.1	8.5		12		15	17	19	21
4	9.1	10	12	13		17		20	22
5	11	12	14	15		18	20	22	24
6	13	14	16			20	22	24	25
7	15	16	18	19	20	22	24	25	27

(f) Đường đi ngắn nhất

Hình 5.1: Các bước thực hiện thuật toán A^*

Lưu đồ thuật toán A^* được minh họa trong Hình 5.2.



Hình 5.2: Lưu đồ thuật toán Astar

5.3 Thuật toán RRT

Thuật toán cây ngẫu nhiên khám phá nhanh RRT là một cấu trúc dữ liệu, được thiết kế để tìm kiếm đường đi một cách hiệu quả trong không gian chiều cao không lỗi. RRT được xây dựng tăng dần bằng cách mở rộng cây đến một điểm được lấy mẫu ngẫu nhiên trong không gian cấu hình trong khi thỏa mãn các ràng buộc đã cho, ví dụ: kết hợp các chương ngại vật hoặc các ràng buộc động (nonholomic hoặc kinodynamic).

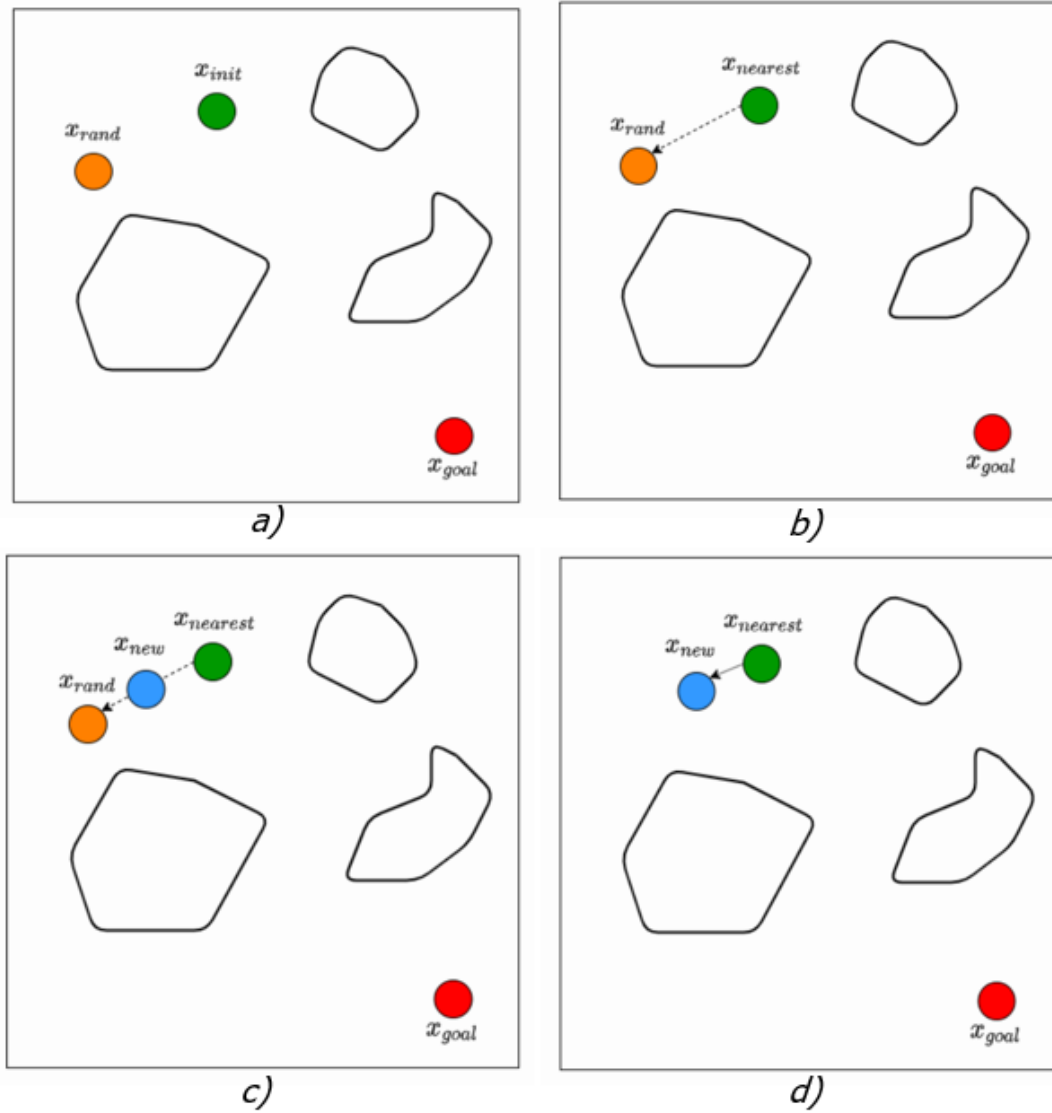
Các bước chạy thuật toán RRT:

Bước 1: Lấy mẫu ngẫu nhiên một điểm x_{rand} trên bản đồ.

Ở bước này, cây bắt đầu từ trạng thái ban đầu x_{init} và mở rộng để tìm đường đi tới trạng thái mục tiêu x_{goal} . Cây sẽ dần dần mở rộng sau mỗi lần tạo điểm ngẫu nhiên. Trong mỗi lần lặp, một trạng thái ngẫu nhiên x_{rand} sẽ được trong không gian.

Bước 2: Tìm điểm lân cận gần nhất $x_{nearest}$ từ cây.

Bước 3: Mở rộng để có được nút x_{new} , kiểm tra xem có đi qua vật cản không.



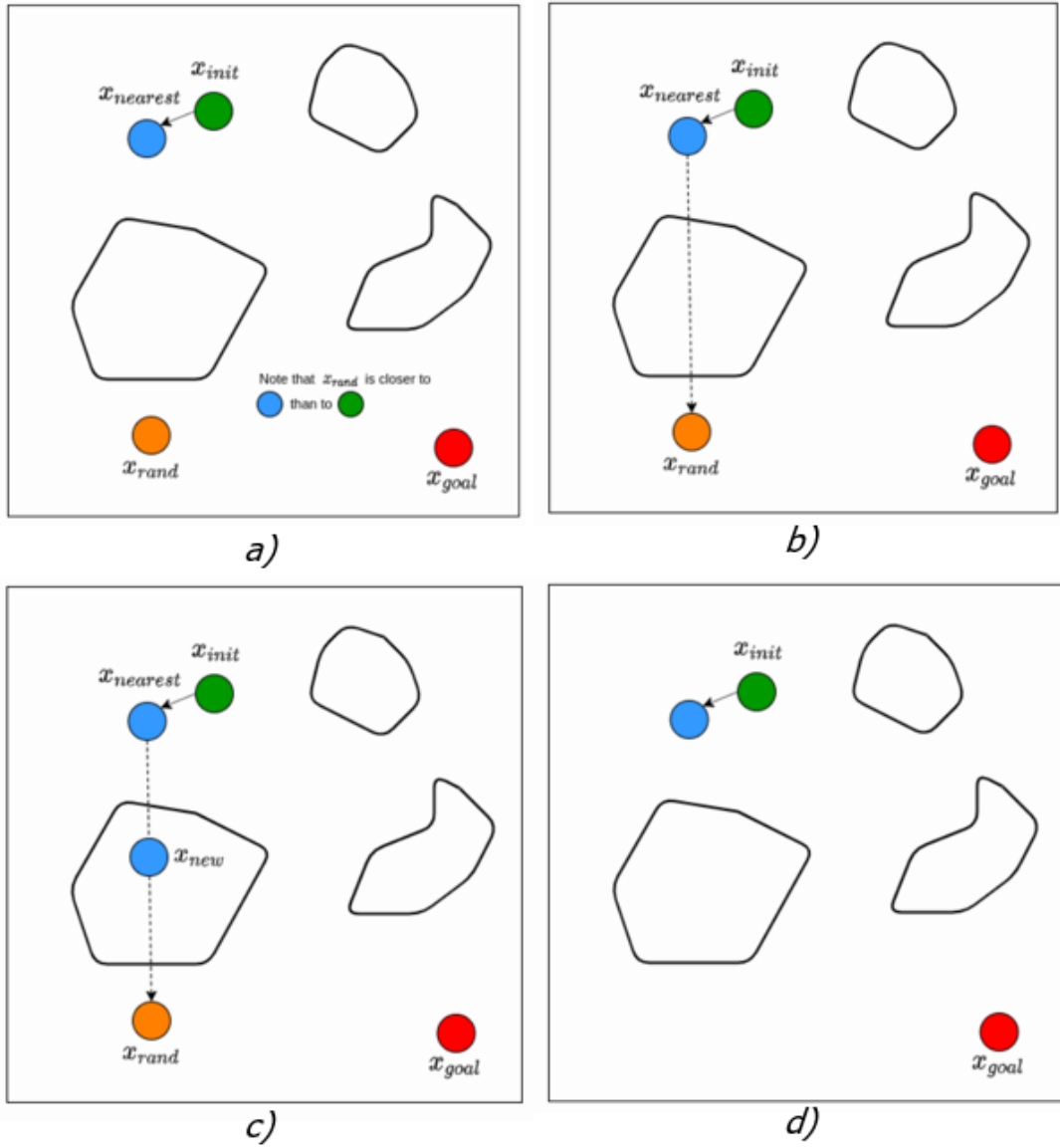
Hình 5.3: Trường hợp x_{new} hợp lệ

Tạo x_{new} nằm trên đường nối giữa $x_{nearest}$ và x_{rand} theo một kích thước ε được xác định trước. Sẽ có 2 trường hợp xảy ra:

- Trường hợp 1: x_{new} nằm trên đường nối và không chứa vật cản thì cây sẽ được mở rộng bằng cách kết nối giữa x_{new} và $x_{nearest}$ như trên hình 5.3.
- Trường hợp 2: x_{new} nằm trên đường nối và đồng thời nằm trong vùng chứa vật cản thì x_{new} sẽ bị loại bỏ như trên hình 5.4. Chu trình sẽ được tiếp tục lại từ bước 1.

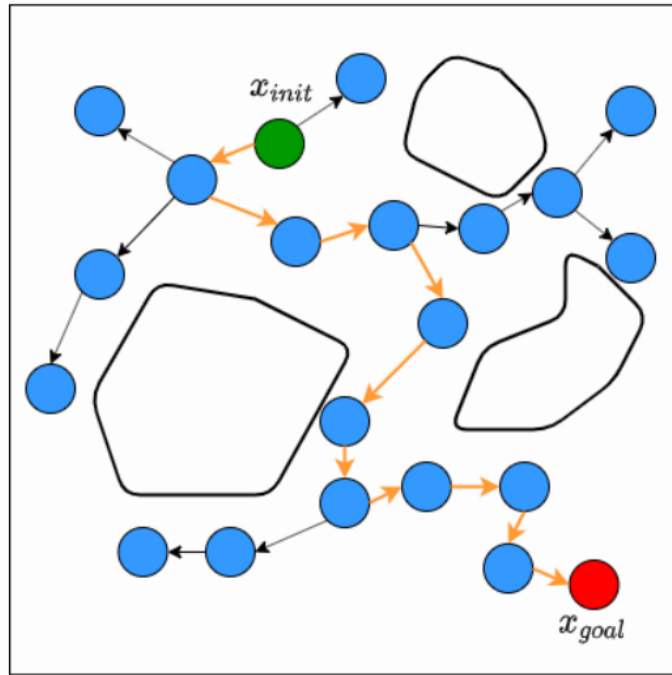
Bước 4: Chèn x_{new} vào cây.

Bước 5: Biểu diễn đường đi giữa x_{init} và x_{new} như Hình 5.5.



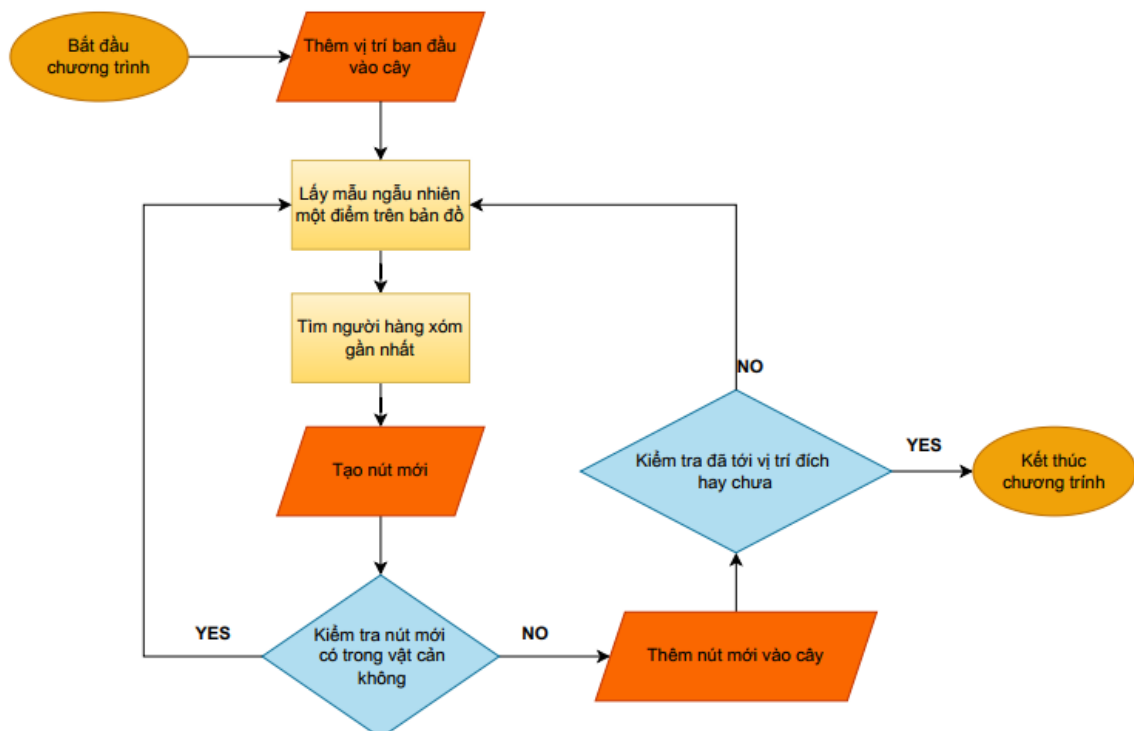
Hình 5.4: Trường hợp x_{new} không hợp hợp lệ

Bước 6: Lặp lại quy trình cho đến khi đạt đến điểm mục tiêu.



Hình 5.5: Đường đi được lập bằng RRT

Lưu đồ thuật toán RRT được minh họa trong Hình 5.6



Hình 5.6: Lưu đồ thuật toán RRT

5.4 Bài tập thực hành

5.4.1 Mô phỏng

Bài tập 5.1: Bật file *Map.ttt* bằng trình mô phỏng VREP. Sau đó mở file *generatemap.m* bằng ứng dụng MATLAB hoàn thiện file code bằng ngôn ngữ lập trình matlab nhằm mục đích xây dựng lại bản đồ trong trình mô phỏng VREP thành bản đồ trong không gian 2D.

Bài tập 5.2: Sinh viên hoàn thiện các phần code còn thiếu trong các file *Atar.m*, *RRT.m* theo từng mục đã được chú thích. Bản đồ là bản đồ đã được tạo từ bài tập 1. Sau khi tìm được tập hợp các điểm để robot cần đi, tiến hành chạy file *testAstar.m* và *testRRT.m* kiểm tra xem Robot trong VREP đã đi được đúng theo quỹ đạo đưa ra hay chưa.

5.4.2 Thực nghiệm

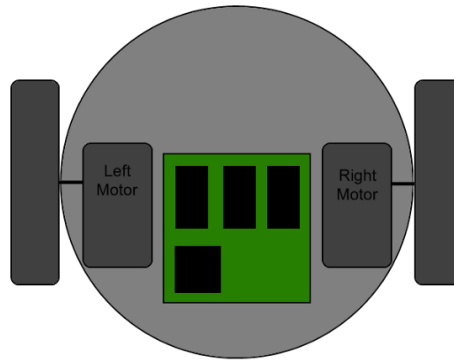
Bài tập 5.3: Tương tự như trong phần mô phỏng, sinh viên sẽ tự xây dựng một bản đồ thực tế ở bên ngoài thành bản đồ không gian 2D thông qua file *generatemap.m*

Bài tập 5.4: Sau khi thu thập được tập hợp điểm robot phải di chuyển. Sử dụng file *move2goal* của phần trước để robot di chuyển theo quỹ đạo. Kiểm tra và đánh giá độ chính xác về vị trí của robot.

Lưu ý: các bài tập thực nghiệm cần sử dụng hệ thống định vị trong nhà Marvemind để cung cấp vị trí cho robot trong mỗi bước chạy.

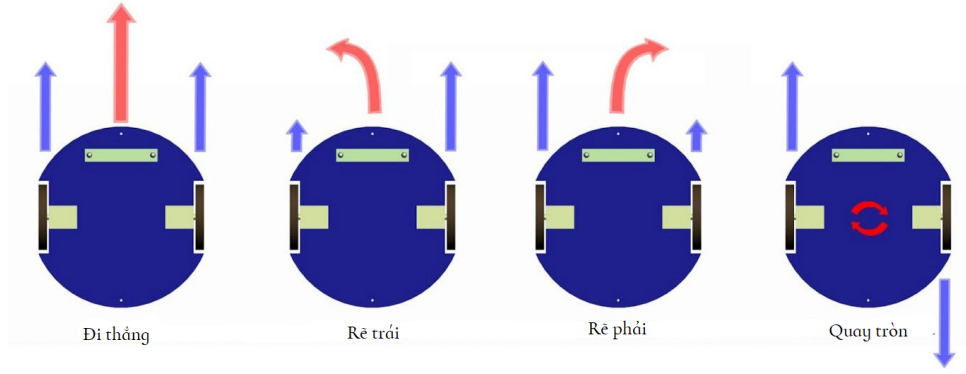
MÔ HÌNH HAI BÁNH VI SAI

A.1 Mô hình hai bánh vi sai



Hình A.1: Mô hình hai bánh vi sai

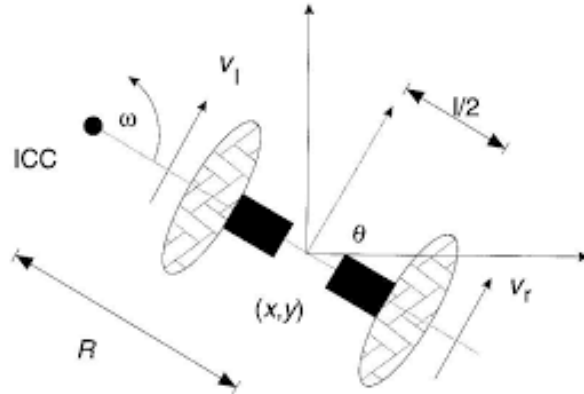
Một robot hai bánh vi sai là một robot di động có chuyển động dựa trên hai bánh xe được điều khiển riêng biệt được đặt ở hai bên của thân robot. Do đó, nó có thể thay đổi hướng của nó bằng cách thay đổi tốc độ xoay tương đối của bánh xe của nó. Đây là dạng hệ thống chuyển động được sử dụng phổ biến nhất trong robot vì nó đơn giản và dễ thực hiện nhất. Nó có bánh xe di chuyển tự do ở phía trước cùng với bánh xe trái và phải. Hai bánh xe được điều khiển bởi các động cơ khác nhau. Tính đơn giản và dễ sử dụng khiến nó trở thành hệ thống được ưa thích nhất bởi những người mới bắt đầu Truyền động độc lập gây khó khăn cho chuyển động theo đường thẳng. Sự khác biệt về động cơ và cấu hình ma sát của hai bánh xe khiến chúng di chuyển với hiệu ứng quay nhẹ. Nhược điểm trên phải được khắc phục bằng hệ thống phản hồi thích hợp. Thích hợp cho robot từ xa do con người điều khiển. Để quay robot sang trái hoặc phải, các bánh xe được quay với tốc độ khác nhau hoặc theo các hướng khác nhau. Mô hình hai bánh vi sai



Hình A.2: Các trường hợp chuyển động của robot hai bánh vi sai

được thể hiện trong hình A.1. Theo hình A.1, khi ta cung cấp vận tốc khác nhau vào mỗi động cơ sẽ tạo nên mỗi chuyển động mới cho robot hai bánh vi sai được biểu diễn trong hình A.2

Mặc dù hai bánh có thể thay đổi với tốc độ khác nhau, để robot thực hiện chuyển động lăn, robot phải quay quanh một điểm nằm dọc trục bánh xe chung bên trái và bên phải của chúng. Điểm mà robot quay quanh được gọi là Tâm cong tức thời (ICC - Instantaneous Center of Curvature). Tâm cong tức thời được mô tả bằng hình A.3



Hình A.3: Động học robot hai bánh vi sai [thêm trích dẫn báo]

Bằng cách thay đổi vận tốc của hai bánh xe, chúng ta có thể thay đổi quỹ đạo mà robot đi. Vì tốc độ quay ω quanh tâm cong tức thời phải như nhau đối với cả hai bánh nên chúng ta có thể viết các phương trình sau:

$$V_r = \omega(R + l/2) \quad (\text{A.1})$$

$$V_l = \omega(R - l/2) \quad (\text{A.2})$$

Trong đó l là khoảng cách giữa tâm của hai bánh xe, V_r , V_l là vận tốc của bánh xe bên phải và bên trái dọc theo mặt đất và R là khoảng cách từ ICC đến điểm giữa của các bánh xe.

Lưu ý rằng robot hai bánh vi sai không thể di chuyển theo hướng dọc theo trục - đây là một điểm kỳ dị. Robot hai bánh vi sai rất nhạy cảm với những thay đổi nhỏ về vận tốc ở mỗi bánh xe. Các lỗi nhỏ về vận tốc tương đối giữa các bánh xe có thể ảnh hưởng đến quỹ đạo của robot.